



Zusammenfassung der Konzepte

- **Lambda-Ausdruck** Ein Lambda-Ausdruck ist ein Codesegment, das für eine spätere Ausführung gespeichert werden kann.
- **Funktionaler Stil** Im funktionalen Stil der Verarbeitung von Sammlungen entnehmen wir nicht jedes Element einzeln, um damit zu arbeiten. Stattdessen übergeben wir der Sammlung ein Codesegment, das auf jedes Element angewandt wird.
- **Streams** Streams vereinheitlichen die Verarbeitung von Elementen einer Sammlung und anderer Datenmengen. Ein Stream bietet nützliche Methoden, um diese Datenmengen zu manipulieren.
- **filter** Wir können eine **filter**-Operation auf einen Stream anwenden, um nur bestimmte Elemente auszuwählen.
- **map** Wir können einen Stream mithilfe der **map**-Operation auf einen anderen Stream abbilden, wobei jedes Element des ursprünglichen Streams durch ein neues Element ersetzt wird, das aus dem Original abgeleitet ist.
- **reduce** Wir können einen Stream mithilfe der **reduce**-Operation reduzieren, das heißt, wir wenden eine Funktion an, die einen ganzen Stream entgegennimmt und einen einzelnen Ergebniswert zurückgibt.
- **Pipeline** Eine Pipeline ist die Kombination von zwei oder mehreren Stream-Funktionen in einer Kette, wobei jede Funktion der Reihe nach angewendet wird.

Übung 5.29 Erstellen Sie eine Kopie des Projekts *Musiksammlung-v5* aus Kapitel 4. Schreiben Sie die Methoden `alleTracksAusgeben` und `bestimmteTracksAusgeben` der Klasse `MusikSammlung` neu, um Streams und Lambda-Ausdrücke zu verwenden.

KAPITEL

6

Bibliotheksklassen nutzen



Lernziele

Zentrale Konzepte in diesem Kapitel: Mit Bibliotheksklassen umgehen, Dokumentation lesen, Dokumentation schreiben

Java-Konstrukte in diesem Kapitel: `String`, `Random`, `HashMap`, `HashSet`, `Iterator`, `static`, `final`, Autoboxing, Wrapper-Klassen

In Kapitel 4 haben wir die Klasse `ArrayList` aus der Java-Klassenbibliothek vorgestellt. Wir haben gesehen, dass wir mit dieser Klasse Dinge tun können, die sonst nur recht schwierig umzusetzen wären (in unserem Beispiel eine beliebige Anzahl von Objekten speichern).

Dies war nur ein Beispiel für eine nützliche Klasse aus der Java-Bibliothek. Die Bibliothek besteht aus Tausenden von Klassen, von denen viele sehr nützlich für Ihre Arbeit sein können, während Sie vermutlich viele andere niemals benötigen werden.

Für einen guten Java-Programmierer ist es wichtig, kompetent mit der Java-Bibliothek arbeiten zu können. Vor allem die Fähigkeit, die richtigen Klassen aus ihr auszuwählen, ist unerlässlich. Sobald Sie mit der Bibliothek einigermaßen vertraut sind, werden Sie merken, dass Sie vieles sehr viel einfacher und schneller mithilfe der Bibliotheksklassen erledigen können als ohne sie. Der Umgang mit diesen Klassen ist der Schwerpunkt in diesem Kapitel.

Bei den Elementen in der Bibliothek handelt es sich nicht um einen Satz von beliebigen Klassen, die in keinem Zusammenhang stehen und die wir alle einzeln lernen müssen, sondern sie werden oft nach bestimmte Kriterien zusammengestellt, die auf Gemeinsamkeiten beruhen. Auch hier haben wir es wieder mit dem Konzept der Abstraktion zu tun, die uns hilft, eine Fülle an Informationen zu verarbeiten. Zu den wichtigsten Teilen der Bibliothek gehören die Sammlungen, zu denen auch die `ArrayList` gehört. Wir werden in diesem Kapitel weitere Sammlungen besprechen und feststellen, dass sie viele Attribute gemeinsam haben, sodass wir oft von den individuellen Details einer speziellen Sammlung abstrahieren und über Sammlungsklassen im Allgemeinen sprechen können.

Wir werden neue Sammlungsklassen sowie einige andere nützliche Bibliotheksklassen vorstellen und diskutieren. Im Verlauf dieses Kapitels werden wir uns mit der Erstellung einer einzelnen Anwendung beschäftigen (einem System für einen

technischen Kundendienst), die viele verschiedene Bibliotheksklassen benutzt. Eine komplette Implementierung mit allen Entwurfsideen und Quelltexten, die wir hier betrachten, samt einiger Zwischenversionen finden Sie in den Buchprojekten. Obwohl Sie dadurch die komplette Lösung untersuchen können, schlagen wir vor, dass Sie die Übungen in diesem Kapitel Schritt für Schritt nachvollziehen. Nachdem wir einen kurzen Blick auf das komplette System geworfen haben, werden wir von einer sehr einfachen Basisversion ausgehen und dann nach und nach die komplette Lösung entwerfen und implementieren.

Die Anwendung benutzt mehrere Bibliotheksklassen und viele für Sie neue Techniken, die alle für sich einige Aufmerksamkeit erfordern: Hash-Abbildungen, Mengen, Zeichenkettenzerlegung und die weitere Verwendung von Zufallszahlen. Sie sollten sich darauf einstellen, dass Sie dieses Kapitel nicht innerhalb eines Tages lesen und verstehen können, sondern dass es etliche Abschnitte enthält, für die Sie einige Tage intensiven Studiums investieren müssen. Dafür werden Sie am Ende, wenn Sie alle Übungen programmiert haben, eine Vielzahl wichtiger Konzepte gelernt haben.

6.1 Die Dokumentation der Bibliotheksklassen

Konzept

Die **Standard-klassenbibliothek** von Java enthält viele Klassen, die sehr nützlich sind. Es ist wichtig zu wissen, wie man die Bibliothek benutzen kann.

Die Standardklassenbibliothek von Java ist riesig. Sie besteht aus Tausenden von Klassen, jede mit vielen Methoden, mit oder ohne Parameter und mit oder ohne Ergebnistypen. Es ist unmöglich, sie alle mitsamt den Details ihrer Benutzung zu lernen. Ein guter Java-Programmierer sollte stattdessen

- einige der wichtigsten Klassen der Bibliothek und ihre Methoden namentlich kennen (**ArrayList** ist eine dieser wichtigen) und
- wissen, wie er sich die anderen Klassen (mitsamt den Details ihrer Methoden und Parameter) ansehen kann.

In diesem Kapitel werden wir einige der wichtigsten Klassen der Bibliothek vorstellen, weitere werden in späteren Kapiteln behandelt werden. Aber viel wichtiger ist, dass Sie auch lernen, wie Sie sich in der Bibliothek allein zurechtfinden können. Dies wird Ihnen ermöglichen, sehr viel interessantere Programme zu schreiben. Glücklicherweise ist die Java-Bibliothek recht gut dokumentiert. Diese Dokumentation liegt im HTML-Format vor, sodass sie mit einem Webbrowser betrachtet werden kann. Wir werden dies für unsere Erkundung der Bibliotheksklassen ausnutzen.

Lesen und Verstehen der Dokumentation ist der erste Teil unserer Einführung in die Bibliotheksklassen. Wir gehen dann noch einen Schritt weiter, indem wir auch diskutieren, wie Sie eigene Klassen so aufbereiten können, dass andere sie genauso benutzen können, als wären sie Standardbibliotheksklassen. Dies ist für die reale Softwareentwicklung sehr wichtig, bei der über einen längeren Zeitraum in Teams an großen Projekten oder in der Softwarewartung gearbeitet wird.

Bei der Benutzung der Klasse **ArrayList** ist Ihnen möglicherweise aufgefallen, dass wir sie einsetzen konnten, ohne einen Blick auf ihren Quelltext geworfen zu haben. Wir haben nicht überprüft, wie sie implementiert ist. Dies war nicht notwendig, um ihre Dienste in Anspruch zu nehmen. Wir kannten lediglich den Namen der Klasse, die Namen der Methoden mit ihren Parametern und Ergebnistypen und wussten,

was die Methoden tun. Wir haben uns nicht dafür interessiert, wie sie ihre Aufgaben erfüllen. Dies ist typisch für die Benutzung von Bibliotheksklassen.

Dies gilt in gleicher Weise für andere Klassen in großen Softwareprojekten. Typischerweise kooperieren mehrere Entwickler in einem Projekt, indem sie an unterschiedlichen Teilen des Systems arbeiten. Jeder Entwickler sollte sich auf seinen Bereich konzentrieren und nicht die Details der anderen Teile verstehen müssen (wir haben dies bereits in Abschnitt 3.2 diskutiert, als wir über Abstraktion und Modularisierung gesprochen haben). Letztlich sollte jeder Entwickler die Klassen der anderen Teammitglieder so benutzen können, als ob sie Bibliotheksklassen wären: also ihre Dienste in Anspruch nehmen, ohne die Details der Realisierung kennen zu müssen.

Damit dies funktioniert, muss jedes Teammitglied seine Klassen in der Art dokumentieren, wie die Standardbibliothek dokumentiert ist. Diese Dokumentation ermöglicht es dann, die Klasse zu benutzen, ohne den Quelltext zu lesen. Auch dieses Thema werden wir in diesem Kapitel besprechen.

6.2 Das Kundendienstsystem

Wie immer werden wir diese Themen anhand eines Beispiels diskutieren. Diesmal verwenden wir das *Kundendienstsystem*. Sie finden das Projekt im Begleitmaterial unter dem Namen *Technischer-Kundendienst1*.

Das *Kundendienstsystem* ist ein Programm, das den technischen Kundendienst für die Kunden der fiktiven Softwarefirma SeltsamSoft abwickeln soll. Noch vor einiger Zeit hatte SeltsamSoft eine Abteilung für den technischen Kundendienst, in der Mitarbeiter an Telefonen saßen. Kunden konnten dort anrufen, um Rat und Hilfe bei technischen Problemen mit den Softwareprodukten von SeltsamSoft zu bekommen. In letzter Zeit ist das Geschäft allerdings nicht mehr sehr gut gelaufen und SeltsamSoft hat aus Kostengründen beschlossen, die Abteilung aufzulösen. Sie wollen nun ein *Kundendienstsystem* entwickeln, das den Eindruck vermitteln soll, dass noch immer Kundendienst angeboten wird. Das System soll die Antworten simulieren, die ein Mitarbeiter im technischen Kundendienst geben könnte. Die Kunden können mit dem System online kommunizieren.

6.2.1 Das Kundendienstsystem erkunden

Übung 6.1 Öffnen und starten Sie das Projekt *Technischer-Kundendienst-komplett*. Sie starten es, indem Sie ein Objekt der Klasse **Kundendienstsystem** erzeugen und seine **starten**-Methode aufrufen. Geben Sie einige Fragen zu Problemen ein, die Sie mit Ihrer Software haben könnten, um das System auszuprobieren. Beobachten Sie, wie es sich verhält. Tippen Sie „ade“ (es ist ein schwäbisches System!), wenn Sie fertig sind. Sie brauchen an dieser Stelle noch nicht den Quelltext zu untersuchen. Dieses Projekt enthält die komplette Lösung, wie wir sie am Ende dieses Kapitels erstellt haben wollen. Der Zweck dieser Übung ist lediglich, dass Sie einen Eindruck vom Zielsystem bekommen.

Eliza

Die Idee für das Kundendienstsystem stammt von dem bahnbrechenden Programm der Künstlichen Intelligenz, Eliza, das von Joseph Weizenbaum am Massachusetts Institute of Technology in den 1960er Jahren entwickelt wurde. Sie können mehr darüber erfahren, indem Sie im Web nach „Eliza“ und „Weizenbaum“ suchen.

Wir werden nun unsere ausführlichere Untersuchung beginnen, indem wir das Projekt *Technischer-Kundendienst1* öffnen. Es enthält eine erste rudimentäre Implementierung unseres Systems. Wir werden es im Laufe dieses Kapitels ausbauen. Auf diese Weise kommen wir zu einem besseren Verständnis des Gesamtsystems, als wenn wir nur die komplette Lösung betrachten würden.

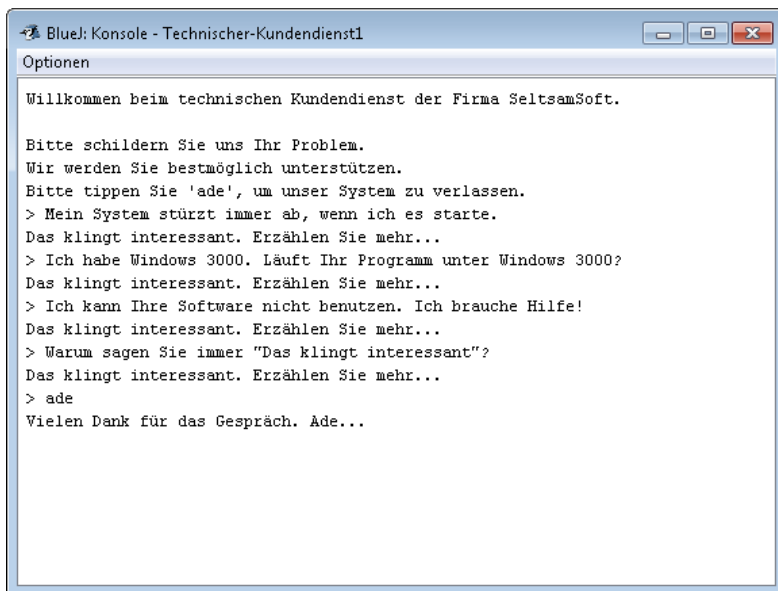
Aus **Übung 6.1** konnten Sie ersehen, dass das System im Kern einen Dialog mit dem Benutzer führt. Der Benutzer kann eine Frage eingeben und das System antwortet. Versuchen Sie nun das Gleiche mit unserem Prototyp des Projekts, *Technischer-Kundendienst1*.

In der kompletten Version schafft es das System, einigermaßen abwechslungsreiche Antworten zu geben. Manchmal scheinen sie sogar sinnvoll zu sein! In dieser ersten Version, die wir weiterentwickeln werden, sind die Reaktionen sehr viel eingeschränkter (Abbildung 6.1). Sie werden schnell feststellen, dass es immer die gleiche Antwort gibt:

Das klingt interessant. Erzählen Sie mehr ...

Abbildung 6.1

Ein erster Dialog mit dem Kundendienstsystem.



Das ist tatsächlich überhaupt nicht interessant und auch nicht sehr überzeugend, wenn eigentlich der Eindruck entstehen soll, dass am anderen Ende der Leitung ein Mitarbeiter des technischen Kundendienstes sitzt. Wir werden bald versuchen, diese Situation zu verbessern. Allerdings sollten wir zuerst etwas genauer untersuchen, was wir bisher vorliegen haben.

Das Projektdiagramm zeigt uns drei Klassen: **Kundendienstsystem**, **Eingabeleser** und **Beantworter** (Abbildung 6.2). **Kundendienstsystem** ist die Hauptklasse, die einen **Eingabeleser** für Eingaben von der Tastatur benutzt und einen **Beantworter**, der Antworten erzeugt.

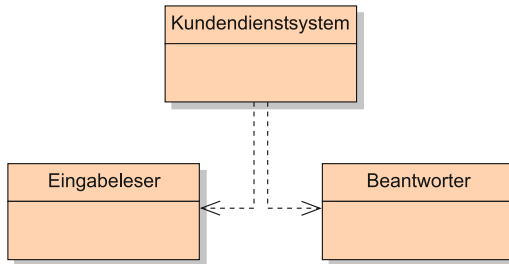


Abbildung 6.2
Das Klassendiagramm
von *Technischer-
Kundendienst*.

Untersuchen Sie den **Eingabeleser** genauer, indem Sie ein Objekt dieser Klasse erzeugen und dann die Objektmethoden betrachten. Sie werden sehen, dass es nur eine Methode anbietet, **gibEingabe**, die eine Zeichenkette zurückliefert. Probieren Sie sie aus. Diese Methode lässt auf der Konsole eine Zeile eingeben und liefert Ihre Eingabe als Aufrufergebnis zurück. Wir werden vorläufig nicht im Detail untersuchen, wie dies funktioniert, sondern lediglich festhalten, dass ein **Eingabeleser** eine Methode **gibEingabe** hat, die eine Zeichenkette zurückliefert.

Machen Sie nun das Gleiche mit der Klasse **Beantworter**. Sie werden feststellen, dass sie eine Methode **generiereAntwort** hat, die immer die Zeichenkette **"Das klingt interessant. Erzählen Sie mehr ..."** liefert. Das erklärt, was wir vorher im Dialog beobachtet haben.

Lassen Sie uns nun einen genaueren Blick auf die Klasse **Kundendienstsystem** werfen.

6.2.2 Den Quelltext untersuchen

Der komplette Quelltext der Klasse **Kundendienstsystem** ist in Listing 6.1 zu sehen. Listing 6.2 zeigt den Quelltext der Klasse **Beantworter**.

Listing 6.1

Der Quelltext
der Klasse **Kunden-**
dienstsystem.

```

/**
 * Diese Klasse implementiert ein System für einen technischen
 * Kundendienst. Das System kommuniziert mit dem Benutzer über
 * die Konsole.
 *
 * Diese Klasse benutzt ein Exemplar der Klasse Eingabeleser,
 * um Eingaben vom Benutzer zu lesen, und ein Exemplar der
 * Klasse Beantworter, um Antworten zu generieren.
 * In einer Schleife werden Eingaben eingelesen und Antworten
 * auf diese Eingaben generiert, bis der Benutzer das System
 * verlässt.
 *
 * @author    Michael Kölling und David J. Barnes
 * @version   0.1 (2016.02.29)
 */
public class Kundendienstsystem
{
    private Eingabeleser leser;
    private Beantworter beantworter;

    /**
     * Erzeuge ein Kundendienstsystem.
     */
    public Kundendienstsystem()
    {
        leser = new Eingabeleser();
        beantworter = new Beantworter();
    }

    /**
     * Starte das System für den technischen Kundendienst.
     * Es wird ein Begrüßungstext ausgegeben und anschließend
     * ein Dialog mit dem Benutzer geführt, bis der Benutzer
     * den Dialog beendet.
     */
    public void starten()
    {
        boolean fertig = false;

        willkommenstextAusgeben();

        while(!fertig) {
            String eingabe = leser.gibEingabe();

            if(eingabe.startsWith("ade")) {
                fertig = true;
            }
            else {
                String antwort = beantworter.generiereAntwort();
                System.out.println(antwort);
            }
        }
        abschiedstextAusgeben();
    }

    /**
     * Gib einen Willkommenstext auf der Konsole aus.
     */
    private void willkommenstextAusgeben()
    {
        System.out.println("Willkommen beim technischen Kundendienst" +
            " der Firma SeltsamSoft.");
        System.out.println();
        System.out.println("Bitte schildern Sie uns Ihr Problem.");
        System.out.println("Wir werden Sie bestmöglich unterstützen.");
        System.out.println("Bitte tippen Sie 'ade', um unser System zu" +
            " verlassen.");
    }

    /**
     * Gib einen Abschiedstext auf der Konsole aus.
     */
    private void abschiedstextAusgeben()
    {
        System.out.println("Vielen Dank für das Gespräch. Ade ...");
    }
}

```



```

/**
 * Die Klasse Beantworter beschreibt Exemplare, die
 * automatische Antworten generieren.
 *
 * @author Michael Kölling und David J. Barnes
 * @version 0.1 (2016.02.29)
 */
public class Beantworter
{
    /**
     * Erzeuge einen Beantworter - nichts Besonderes zu tun.
     */
    public Beantworter()
    {
    }

    /**
     * Erzeuge eine Antwort.
     * @return einen String, der die Antwort enthält
     */
    public String generiereAntwort()
    {
        return "Das klingt interessant. Erzählen Sie mehr ...";
    }
}

```

Listing 6.2

Der Quelltext der Klasse **Beantworter**.

Ein Blick auf Listing 6.2 zeigt uns, dass die Klasse **Beantworter** trivial ist. Sie hat nur eine Methode und diese liefert immer dieselbe Zeichenkette. Das ist etwas, was wir später verbessern werden. Vorläufig konzentrieren wir uns auf die Klasse **Kundendienstsystem**.

Kundendienstsystem deklariert zwei Datenfelder, um einen **Eingabeleser** und einen **Beantworter** halten zu können, und initialisiert diese Datenfelder mit entsprechenden Objekten, die im Konstruktor erzeugt werden.

Am Ende hat die Klasse zwei Methoden, **willkommenstextAusgeben** und **abschiedstextAusgeben**. Diese geben einfach kurze Texte aus – eine Begrüßung und einen Abschiedstext.

Der interessanteste Abschnitt im Quelltext ist die Methode in der Mitte: **starten**. Wir werden diese Methode genauer untersuchen.

Zu Anfang der Methode erfolgt ein Aufruf von **willkommenstextAusgeben**, am Ende steht ein Aufruf von **abschiedstextAusgeben**. Diese Aufrufe sorgen für die entsprechenden Ausgaben zu den geeigneten Zeitpunkten. Der Rest der Methode besteht aus der Deklaration einer booleschen Variablen und einer **while**-Schleife. Die Struktur ist folgendermaßen:

```

boolean fertig = false;
while(!fertig) {
    tue etwas
    if(Ende-Bedingung) {
        fertig = true;
    }
    else {
        tue etwas anderes
    }
}

```

Dieses Programmiermuster ist eine Abwandlung der **while**-Schleife, die wir in Abschnitt 4.10 diskutiert haben. Wir benutzen **fertig** als einen Signalgeber, der

den Wert **true** bekommt, wenn wir die Schleife beenden wollen (und damit das ganze Programm). Zu Anfang stellen wir sicher, dass **fertig** den Wert **false** hat. (Erinnern Sie sich daran, dass ein Ausrufezeichen den *Nicht*-Operator darstellt!)

Der Hauptteil der Schleife – der Teil, der wiederholt ausgeführt wird, solange wir noch nicht fertig sind – besteht aus drei Anweisungen, wenn wir die Prüfung der Ende-Bedingung auslassen:

```
String eingabe = leser.gibEingabe();
...
String antwort = beantworter.generiereAntwort();
System.out.println(antwort);
```

In dieser Schleife wird somit wiederholt

- eine Benutzereingabe gelesen
- der Beantworter um eine Antwort gefragt
- diese Antwort ausgegeben

(Sie werden sicherlich bemerkt haben, dass die Antwort überhaupt nicht von der Eingabe abhängig ist! Das ist etwas, was wir ganz sicher ändern werden.)

Der letzte zu untersuchende Teil ist die Prüfung der Ende-Bedingung. Die Idee ist, dass das Programm enden soll, sobald der Benutzer „ade“ eingegeben hat. Die entsprechende Stelle im Quelltext lautet:

```
String eingabe = leser.gibEingabe();
if(eingabe.startsWith("ade")) {
    fertig = true;
}
```

Wenn Sie diese Abschnitte einzeln verstanden haben, dann sollten Sie noch einmal einen Blick auf die Methode **starten** in Listing 6.1 werfen, um zu sehen, ob Sie auch das Zusammenspiel verstehen.

In dem letzten Quelltextabschnitt, den wir oben betrachtet haben, wird eine Methode **startsWith** benutzt. Da diese Methode an der Variablen **eingabe** aufgerufen wird, die auf ein Objekt vom Typ **String** verweist, muss dies wohl eine Methode der Klasse **String** sein. Aber was tut diese Methode? Und wie finden wir das heraus?

Wir könnten einfach durch Deuten des Namens **startsWith** (englisch für „beginnt mit“) vermuten, dass sie prüft, ob die Zeichenkette mit dem Wort „ade“ beginnt. Wir können dies durch Probieren belegen. Starten Sie das Kundendienstsystem erneut und tippen Sie „ade, ade“ oder „ade“. Sie werden feststellen, dass beide Versionen das System beenden. Beachten Sie aber, dass eine Eingabe wie „Ade“ oder „ ade“ – mit einem Großbuchstaben oder einem Leerzeichen am Anfang – nicht als beginnend mit „ade“ erkannt wird. Das könnte etwas lästig für den Benutzer sein, aber es wird sich herausstellen, dass wir dies recht leicht beheben können, sobald wir etwas mehr über die Klasse **String** wissen.

Wie bekommen wir mehr Informationen über die Methode **startsWith** oder andere Methoden der Klasse **String**?

6.3 Die Klassendokumentation lesen

Die Klasse **String** ist eine Klasse aus der Java-Standardbibliothek. Wir können mehr über sie erfahren, indem wir die Bibliotheksdokumentation der Klasse **String** nachlesen.

Wählen Sie dazu den Eintrag **JAVA KLASSENBIBLIOTHEKEN** aus dem Menü **HILFE** in BlueJ. Dies öffnet einen Webbrowser, der die Hauptseite der Java-API-Dokumentation (API steht für Application Programmer Interface, die Schnittstelle für Anwendungsprogrammierer) anzeigt.¹

Der Webbrowser zeigt drei Bereiche. In der linken oberen Ecke sehen Sie eine Liste der Pakete. Darunter sehen Sie eine Liste aller Klassen in der Java-Bibliothek. Der große Bereich auf der rechten Seite zeigt Detailinformationen über das selektierte Paket oder die selektierte Klasse.

Suchen Sie in der Liste auf der linken Seite nach der Klasse **String** und selektieren Sie sie. Der Bereich auf der rechten Seite zeigt daraufhin die Dokumentation der Klasse **String** an (Abbildung 6.3).

Konzept

Die **Dokumentation der Java-Standardbibliothek** zeigt Informationen über alle Klassen in der Bibliothek. Die Benutzung dieser Dokumentation ist unerlässlich für einen vernünftigen Gebrauch der Bibliotheksklassen.

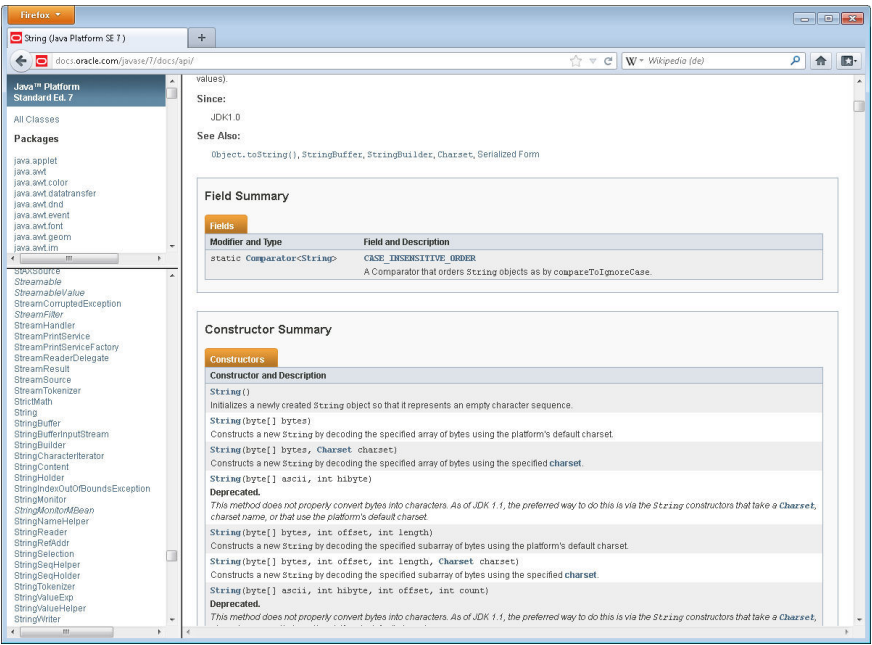


Abbildung 6.3
Dokumentation der Java-Standardbibliothek.

¹ Per Voreinstellung greift diese Funktion auf die Dokumentation im Internet zu. Dies funktioniert nicht, wenn Ihr Computer keinen Netzzugang hat. BlueJ kann so konfiguriert werden, dass es auf eine lokale Kopie der Java-API-Dokumentation zugreift. Dies ist zu empfehlen, da es die Zugriffe beschleunigt und ohne einen Netzzugang funktioniert. Lesen Sie Anhang A für weitere Details.

Übung 6.2 Untersuchen Sie die Dokumentation von **String**. Sehen Sie sich dann die Dokumentation einiger anderer Klassen an. Wie ist die Dokumentation strukturiert? Welche Abschnitte haben alle Klassenbeschreibungen? Welchem Zweck dienen sie?

Übung 6.3 Suchen Sie die Methode **startsWith** in der Dokumentation von **String**. Es gibt sie in zwei Versionen. Beschreiben Sie mit eigenen Worten, was sie tun und welche Unterschiede zwischen ihnen bestehen.

Übung 6.4 Gibt es eine Methode in der Klasse **String**, die testet, ob eine Zeichenkette mit einem gegebenen Suffix endet? Wenn ja, wie heißt sie und welche Parameter und welchen Ergebnistyp hat sie?

Übung 6.5 Gibt es eine Methode in der **String**-Klasse, die die Anzahl der Zeichen in einer Zeichenkette liefert? Wenn ja, wie heißt sie und welche Parameter hat sie?

Übung 6.6 Wenn Sie Methoden für die beiden vorigen Aufgaben gefunden haben, wie haben Sie dies getan? Ist es einfach oder schwierig, nach Methoden zu suchen? Warum?

6.3.1 Schnittstellen versus Implementierungen

Konzept

Die **Schnittstelle** einer Klasse beschreibt, was eine Klasse leistet und wie sie benutzt werden kann, ohne dass ihre Implementierung sichtbar wird.

Sie werden feststellen, dass die Dokumentation unterschiedliche Informationen liefert. Sie enthält, neben anderen Dingen:

- den Namen der Klasse
- eine allgemeine Beschreibung des Zwecks der Klasse
- eine Liste der Konstruktoren und Methoden der Klasse
- die Parameter und Ergebnistypen für jeden Konstruktor und jede Methode
- eine Beschreibung des Zwecks jedes Konstruktors und jeder Methode

Diese Informationen werden zusammengefasst als die *Schnittstelle* der Klasse bezeichnet. Beachten Sie, dass die Schnittstelle *nicht* den Quelltext zeigt, der die Klasse implementiert. Wenn eine Klasse gut beschrieben ist (genauer, wenn ihre Schnittstelle gut beschrieben ist), dann braucht ein Programmierer den Quelltext der Klasse nicht zu sehen, um die Klasse zu benutzen. Die Schnittstelle liefert alle notwendigen Informationen. Dies ist wieder einmal ein Fall von Abstraktion in Aktion.

Konzept

Der komplette Quelltext, der eine Klasse definiert, wird als die **Implementierung** dieser Klasse bezeichnet.

Der Quelltext hinter den Kulissen, der die Klasse ihren Dienst erfüllen lässt, wird als die *Implementierung* der Klasse bezeichnet. Normalerweise arbeitet ein Programmierer an der Implementierung von genau einer Klasse, während er einige andere Klassen über ihre Schnittstellen benutzt.

Die Unterscheidung zwischen der Schnittstelle und der Implementierung einer Klasse ist ein sehr wichtiges Konzept, das uns in diesem und den folgenden Kapiteln immer wieder begegnen wird.

Hinweis

Der Begriff *Schnittstelle* hat mehrere Bedeutungen im Bereich der Programmierung mit Java. Er wird benutzt, um den öffentlich sichtbaren Anteil einer Klasse zu beschreiben (so haben wir ihn gerade kennengelernt), hat aber auch noch andere Bedeutungen: Die Benutzungsschnittstelle (oft eine grafische Benutzungsschnittstelle) wird auch häufig einfach nur als *die Schnittstelle* bezeichnet. Weiterhin verfügt Java noch über ein Sprachkonstrukt namens Interface, das ins Deutsche übersetzt auch Schnittstelle heißt (dieses wird in Kapitel 12 besprochen). Wenn wir über das Java-Sprachkonstrukt reden, werden wir den englischen Begriff verwenden.

Es ist wichtig, die unterschiedlichen Bedeutungen des Begriffs *Schnittstelle* in den verschiedenen Zusammenhängen auseinanderhalten zu können.

Der Begriff der Schnittstelle ist auch auf einzelne Methoden anwendbar. Beispielsweise zeigt uns die Dokumentation der Klasse **String** die Schnittstelle der Methode **length**:

```
public int length()
```

Returns the length of this string. The length is equal to the number of Unicode code units in the string.

Specified by:

length in interface CharSequence

Returns:

the length of the sequence of characters represented by this object.

Die Schnittstelle einer Methode besteht aus dem *Kopf* der Methode und einem Kommentar (der hier kursiv angegeben ist). Zum Kopf einer Methode gehören (in dieser Reihenfolge):

- ein Zugriffsmodifikator (hier **public**) – wir werden diesen noch diskutieren
- der Ergebnistyp der Methode (hier **int**)
- der Name der Methode
- eine Liste von Parametern (die in diesem Beispiel leer ist); der Name und die Parameter werden zusammen auch als die *Signatur* der Methode bezeichnet.

Die Schnittstelle enthält all die Informationen, die wir für die Benutzung dieser Methode benötigen.

6.3.2 Methoden von Bibliotheksklassen benutzen

Zurück zu unserem *Kundendienstsystem*. Wir wollen nun die Verarbeitung der Eingabe etwas verbessern. Wir haben in der obigen Diskussion festgestellt, dass sich das System nicht sehr tolerant verhält. Wenn wir beispielsweise „Ade“ oder „ade“

statt „ade“ eintippen, wird das Wort nicht erkannt. Eine Sache, die wir beachten müssen, ist, dass ein **String**-Objekt nach seiner Erzeugung nicht mehr geändert werden kann. Wir wollen dies ändern, indem wir den vom Benutzer eingegebenen Text so anpassen, dass all diese Variationen als „ade“ erkannt werden.

Die Dokumentation der Klasse **String** gibt uns die Auskunft, dass sie über eine Methode **trim** verfügt, mit der Leerzeichen zu Beginn und am Ende einer Zeichenkette abgeschnitten werden können. Wir können diese Methode verwenden, um den zweiten Problemfall zu behandeln.

Übung 6.7 Finden Sie die Methode **trim** in der Dokumentation der Klasse **String**. Schreiben Sie den Kopf dieser Methode auf. Schreiben Sie ein Beispiel für den Aufruf dieser Methode an einer **String**-Variablen **text** auf.

Konzept

Ein Objekt wird als **unveränderlich** bezeichnet, wenn sein Inhalt oder Zustand nicht mehr geändert werden kann, nachdem es erzeugt wurde. **String** ist ein Beispiel für eine Klasse, die unveränderliche Objekte definiert.

Ein wichtiges Merkmal von **String**-Objekten ist, dass sie *unveränderlich* sind – das heißt, sie können nicht mehr geändert werden, nachdem sie erzeugt wurden. Beachten Sie vor allem, dass die Methode **trim** beispielsweise eine neue Zeichenkette zurückliefert und nicht die ursprüngliche Zeichenkette ändert. Deshalb sollten Sie den nachfolgenden Einschub mit den Fallstricken sorgfältig lesen.

Fallstrick

Häufig wird in Java der Fehler gemacht, eine Zeichenkette verändern zu wollen – zum Beispiel durch folgende Anweisung

```
eingabe.toUpperCase();
```

Dies ist nicht korrekt (Zeichenketten können nicht geändert werden), erzeugt aber leider keinen Fehler. Die Anweisung hat einfach keine Auswirkung und die Eingabezeichenkette bleibt unverändert.

Die Methode **toUpperCase** sowie andere **String**-Methoden ändern nicht die ursprüngliche Zeichenkette, sondern liefern stattdessen eine *neue* Zeichenkette zurück, die gleich der alten ist einschließlich einiger Änderungen (hier mit den Zeichen, die in Großbuchstaben geändert wurden). Wenn wir wollen, dass unsere Variable **eingabe** geändert wird, dann müssen wir dieses neue Objekt wie folgt wieder der Variablen zuweisen (wobei wir uns der alten Variablen entledigen):

```
eingabe = eingabe.toUpperCase();
```

Das neue Objekt könnte auch einer anderen Variablen zugewiesen oder anderweitig verarbeitet werden.

Nachdem wir uns die Schnittstelle der Methode **trim** angesehen haben, wissen wir, dass wir die Leerzeichen aus der Eingabe mit folgendem Aufruf entfernen können:

```
eingabe = eingabe.trim();
```

Diese Anweisung veranlasst das **String**-Objekt, auf das **eingabe** verweist, eine neue Zeichenkette zu erzeugen, in der führende und anhängende Leerzeichen entfernt sind. Diese neue Zeichenkette wird dann in der Variablen **eingabe** gespeichert, da wir die alte nicht mehr benötigen. Somit verweist **eingabe** nach Ausführung dieser Zeile auf eine Zeichenkette, die an beiden Enden keine Leerzeichen mehr hat.

Wir können diese Zeile nun in unseren Quelltext einfügen, sodass er folgendermaßen aussieht:

```
String eingabe = leser.gibEingabe();
eingabe = eingabe.trim();

if(eingabe.startsWith("ade")) {
    fertig = true;
}
else {
    ... Quelltext hier ausgelassen
}
```

Die ersten beiden Zeilen können auch zu einer zusammengefasst werden:

```
String eingabe = leser.gibEingabe().trim();
```

Der Effekt dieser Zeile ist identisch mit den ersten beiden Zeilen oben. Die rechte Seite sollte gelesen werden, als wäre sie folgendermaßen geklammert:

```
(leser.gibEingabe()).trim();
```

Welche Version Sie bevorzugen, ist überwiegend eine Frage des Geschmacks. Die Entscheidung sollte vor allem in Hinsicht auf Lesbarkeit getroffen werden: Benutzen Sie die Version, die Sie einfacher zu lesen und zu verstehen finden. Programmieranfänger bevorzugen häufig die zweizeilige Version, während erfahrene Programmierer sich an die einzeilige Version gewöhnt haben.

Übung 6.8 Implementieren Sie diese Verbesserung in Ihrer Version des Projekts *Technischer-Kundendienst1*. Testen Sie, ob anschließend Leerzeichen vor oder nach „ade“ toleriert werden.

Wir haben nun zwar das Problem mit den Leerzeichen um die Eingabe gelöst, aber das Problem mit den Großbuchstaben besteht weiterhin. Eine weitere Untersuchung der Dokumentation der Klasse **String** führt jedoch zu einer möglichen Lösung, denn dort ist eine Methode **toLowerCase** beschrieben.

Übung 6.9 Verbessern Sie die Implementierung der Klasse **Kundendienstsystem** im Projekt *Technischer-Kundendienst1* so, dass die Groß-/Kleinschreibung bei der Eingabe ignoriert wird. Benutzen Sie dazu die Methode **toLowerCase** der Klasse **String**. Beachten Sie auch hier wieder, dass diese Methode nicht die Zeichenkette ändert, an der sie aufgerufen wird, sondern dass der Aufruf zur Erzeugung einer neuen mit leicht geändertem Inhalt führt.

6.3.3 Zeichenketten auf Gleichheit prüfen

Eine alternative Lösung wäre gewesen, zu prüfen, ob die eingegebene Zeichenkette die Zeichenkette „ade“ tatsächlich *ist*, und nicht, ob sie lediglich mit *ade* beginnt. Ein (falscher!) Versuch, dies zu formulieren, könnte so aussehen:

```
if (eingabe == "ade") { // funktioniert nicht immer!
    ...
}
```

Das Problem ist, dass es mehrere **String**-Objekte geben kann, die alle denselben Text repräsentieren. Zwei **String**-Objekte könnten beispielsweise beide aus den Zeichen „ade“ bestehen. Der Gleichheits-Operator (**==**) überprüft, ob seine beiden Operanden sich *auf dasselbe Objekt* beziehen, nicht, ob sie den gleichen Wert haben! Dies ist ein wichtiger Unterschied.

In unserem Beispiel wollen wir nicht wissen, ob die Variable **eingabe** und die Zeichenkettenkonstante „ade“ dasselbe Objekt bezeichnen, sondern, ob sie den gleichen Wert haben. Die Benutzung des Operators **==** ist somit falsch. Sie könnte **false** liefern, auch wenn der Wert der Variablen **eingabe** „ade“ ist.²

Die Lösung liegt in der Verwendung der Methode **equals**, die in der Klasse **String** definiert ist. Diese Methode überprüft, ob zwei **String**-Objekte den gleichen Inhalt haben. Der korrekte Quelltext sieht also so aus:

```
if(eingabe.equals("ade")) {
    ...
}
```

Dies kann natürlich auch mit den Methoden **trim** und **toLowerCase** kombiniert werden.

Fallstrick

Das Vergleichen von **String**-Objekten mit dem Operator **==** kann zu unerwarteten Ergebnissen führen. Als Grundregel gilt, dass Zeichenketten praktisch immer mit **equals** statt mit **==** verglichen werden sollten.

Übung 6.10 Finden Sie die Methode **equals** in der Dokumentation der Klasse **String**. Welchen Ergebnistyp hat diese Methode?

Übung 6.11 Ändern Sie Ihre Implementierung so, dass sie die Methode **equals** anstelle von **startsWith** verwendet.

² Leider führt aufgrund von Javas **String**-Implementierung der Vergleich zweier verschiedener **String**-Objekte gleichen Inhalts mit **==** oft irreführenderweise zur „korrekten“ Antwort. Trotzdem sollten Sie zum Vergleichen der Inhalte zweier **String**-Objekte niemals **==** verwenden.

6.4 Zufälliges Verhalten einbringen

Bisher haben wir einige kleine Verbesserungen am Projekt *Technischer-Kundendienst* vorgenommen, aber insgesamt ist das System noch sehr simpel. Eines der Hauptprobleme ist, dass es immer dieselbe Antwort liefert, unabhängig von der Eingabe durch den Benutzer. Wir werden dies nun verbessern, indem wir einige plausible Antworten definieren. Aus diesen Antworten lassen wir zufällig eine auswählen, wenn eine Anfrage gestellt wird. Dazu müssen wir die Klasse **Beantworter** in unserem Projekt erweitern.

Wir gehen folgendermaßen vor: Wir legen einige Antworten in einer **ArrayList** ab, generieren eine Zufallszahl und benutzen diese als Index in der Antwortliste, um eine der Antworten auszuwählen. In dieser Version werden wir die Antwort noch nicht von der Eingabe des Benutzers abhängig machen (das kommt später), aber zumindest sind die Antworten dann etwas abwechslungsreicher und sehen deutlich besser aus.

Dazu müssen wir herausfinden, wie man Zufallszahlen generieren kann.

Zufall und Pseudozufall

Die Erzeugung von Zufallszahlen auf einem Computer ist nicht so einfach, wie man es vermuten könnte. Da Computer nach wohldefinierten deterministischen Verfahren arbeiten, die darauf beruhen, dass alle Berechnungen vorhersagbar und wiederholbar sind, gibt es eigentlich keinen Platz für zufälliges Verhalten in Computern.

Wissenschaftler haben im Laufe der Zeit viele Algorithmen vorgeschlagen, mit denen zufällig wirkende Folgen von Zahlen erzeugt werden können. Diese Zahlen sind tatsächlich nicht zufällig, sondern folgen lediglich sehr komplizierten Regeln. Deshalb werden sie auch Pseudozufallszahlen genannt.

In einer Sprache wie Java wurde die Erzeugung von Pseudozufallszahlen glücklicherweise durch eine Bibliotheksklasse implementiert, sodass wir, um Zufallszahlen zu bekommen, lediglich diese Klasse benutzen müssen.

Wenn Sie mehr zu diesem Thema lesen wollen, dann suchen Sie im Web nach den Schlüsselwörtern „Zufallszahlen Computer“ oder auf Englisch „pseudo random numbers“.

6.4.1 Die Klasse Random

Die Klassenbibliothek von Java enthält eine Klasse **Random** (englisch für „Zufall“), die für unser Projekt nützlich ist.

Übung 6.12 Finden Sie die Klasse **Random** in der Dokumentation der Java-Bibliothek. In welchem Paket ist sie zu finden? Was tut sie? Wie erzeugen Sie eine Instanz? Wie generieren Sie eine Zufallszahl? Möglicherweise verstehen Sie nicht alle Erläuterungen in der Dokumentation. Versuchen Sie nur die Dinge herauszufinden, die Sie unmittelbar benötigen.

Übung 6.13 Versuchen Sie, einen Quelltextabschnitt auf ein Blatt Papier zu schreiben, der Ihnen mithilfe dieser Klasse eine Zufallszahl generiert.

Um eine Zufallszahl zu erzeugen, müssen wir

- eine Instanz der Klasse **Random** erzeugen
- an dieser Instanz eine Methode aufrufen, um eine Zufallszahl zu erhalten

Ein Blick auf die Dokumentation sagt uns, dass es mehrere Methoden **next-Irgendetwas** gibt, die Zufallszahlen verschiedener Typen erzeugen. Diejenige zur Erzeugung von ganzen Zufallszahlen heißt **nextInt**.

Der folgende Quelltextabschnitt zeigt, wie eine ganze Zufallszahl generiert und ausgegeben werden kann:

```
Random zufallsgenerator;  
zufallsgenerator = new Random();  
int index = zufallsgenerator.nextInt();  
System.out.println(index);
```

In diesem Abschnitt wird eine neue Instanz der Klasse **Random** erzeugt und in der Variablen **zufallsgenerator** abgelegt. Anschließend wird an diesem Objekt die Methode **nextInt** aufgerufen, um eine Zufallszahl zu erhalten; diese wird in der Variablen **index** abgelegt und schließlich ausgegeben.

Übung 6.14 Schreiben Sie Quelltext (in BlueJ), der die Erzeugung von Zufallszahlen testet. Definieren Sie dazu eine Klasse **ZufallszahlenTester**. Sie können diese Klasse im Projekt *Technischer-Kundendienst* anlegen oder ein eigenes Projekt dafür anlegen. Implementieren Sie in der Klasse zwei Methoden: **eine-ZufallszahlAusgeben** (die eine Zufallszahl ausgeben soll) und **zufallszahlen-Ausgeben(int anzahl)** (die einen Parameter bekommt, der angibt, wie viele Zufallszahlen generiert und ausgegeben werden sollen).

Beachten Sie, dass Ihre Klasse nur eine Instanz der Klasse **Random** erzeugen und diese in einem Datenfeld ablegen sollte. Erzeugen Sie nicht für jede Zufallszahl eine neue Instanz von **Random**.

6.4.2 Zufallszahlen mit eingeschränktem Wertebereich

Die Zufallszahlen, die wir bisher gesehen haben, liegen im Wertebereich der ganzen Zahlen von Java (von -2147483648 bis 2147483647). Das ist zum Experimentieren ganz okay, aber nicht sehr nützlich. Sehr häufig benötigen wir Zufallszahlen aus einem definierten Wertebereich.

Die Klasse **Random** bietet eine Methode an, die dies unterstützt. Sie heißt ebenfalls **nextInt**, hat jedoch einen Parameter, mit dem der gewünschte Wertebereich angegeben werden kann.

Übung 6.15 Finden Sie die Methode `nextInt` in der Klasse `Random`, mit der Sie den Wertebereich der Zufallszahlen angeben können. Aus welchem Wertebereich stammen die erzeugten Zahlen, wenn Sie die Methode mit dem Parameter 100 aufrufen?

Übung 6.16 Schreiben Sie eine Methode `wuerfeln` in Ihrer Klasse `ZufallszahlenTester`, die Werte von 1 bis einschließlich 6 zurückliefert.

Übung 6.17 Schreiben Sie eine Methode `gibAntwort`, die zufällig eine der Zeichenketten `"ja"`, `"nein"` und `"vielleicht"` zurückliefert.

Übung 6.18 Erweitern Sie Ihre Methode `gibAntwort` so, dass sie eine `ArrayList` mit einer beliebigen Anzahl an Antworten benutzt, aus denen sie zufällig eine auswählt.

Wenn Sie eine Methode zur Erzeugung von Zufallszahlen aus einem bestimmten Wertebereich benutzen, dann sollten Sie sehr genau darauf achten, ob die Grenzen des Wertebereichs als *inklusive* oder *exklusiv* angesehen werden. Die Methode `nextInt(int n)` in der Klasse `Random` beispielsweise spezifiziert, dass sie Zahlen aus dem Bereich von 0 (inklusive) bis `n` (exklusiv) erzeugt. Der Wert 0 ist somit ein mögliches Ergebnis eines Aufrufs, während der angegebene Wert für `n` niemals zurückgeliefert wird. Die höchste gelieferte Zahl ist `n-1`.

Übung 6.19 Fügen Sie Ihrer Klasse `ZufallszahlenTester` eine Methode hinzu, die einen Parameter `max` erhält und eine Zufallszahl aus dem Bereich 1 bis `max` liefert (inklusive).

Übung 6.20 Fügen Sie Ihrer Klasse `ZufallszahlenTester` eine Methode hinzu, die zwei Parameter `min` und `max` erhält und eine Zufallszahl aus dem Bereich von `min` bis `max` liefert (inklusive). Schreiben Sie Ihre Methode aus der vorigen Übung so um, dass sie nun diese neue Methode benutzt, um ihr Ergebnis zu erzeugen. Beachten Sie, dass diese Methode nicht notwendigerweise mithilfe einer Schleife implementiert werden muss.

Übung 6.21 Sehen Sie sich die Details der Klasse `SecureRandom` an, die im `java.security`-Paket definiert ist. Könnte diese Klasse anstelle der `Random`-Klasse verwendet werden? Warum sind Zufallszahlen für kryptografische Sicherheit wichtig?

6.4.3 Zufällige Antworten generieren

Wir können nun zur Klasse `Beantworter` zurückkehren, um diese zufällig eine Antwort aus einer Menge von vorgegebenen Antworten auswählen zu lassen. Listing 6.2 zeigt den Quelltext der Klasse `Beantworter` in unserer ersten Version.

Wir werden diesen Quelltext nun um einige Zeilen anreichern, die

- ein Datenfeld vom Typ `Random` für einen Zufallszahlengenerator deklarieren
- ein Datenfeld vom Typ `ArrayList` für die möglichen Antworten deklarieren

- im Konstruktor von **Beantworter** ein **Random**- und ein **ArrayList**-Objekt erzeugen
- die Antwortliste mit einigen Antworten füllen
- eine Antwort zufällig auswählen und zurückliefern, wenn **generiereAntwort** aufgerufen wird

Listing 6.3 zeigt eine Version der Klasse **Beantworter** mit diesen Ergänzungen.

Listing 6.3

Der Quelltext der Klasse **Beantworter** mit zufälligen Antworten.

```
import java.util.ArrayList;
import java.util.Random;

/**
 * Die Klasse Beantworter beschreibt Exemplare, die
 * automatische Antworten generieren. Dies ist die zweite Version
 * dieser Klasse. In dieser Version generieren wir Antworten,
 * indem wir zufällig eine Antwortphrase aus einer Liste auswählen.
 *
 * @author Michael Kölling und David J. Barnes
 * @version 0.2 (2016.02.29)
 */
public class Beantworter
{
    private Random zufallsgenerator;
    private ArrayList<String> antworten;

    /**
     * Erzeuge einen Beantworter
     */
    public Beantworter()
    {
        zufallsgenerator = new Random();
        antworten = new ArrayList<>();
        antwortlisteFuellen();
    }

    /**
     * Generiere eine Antwort.
     *
     * @return einen String, der die Antwort enthält
     */
    public String generiereAntwort()
    {
        // Erzeuge eine Zufallszahl, die als Index in der Liste der
        // Antworten benutzt werden kann. Die Zahl wird im Bereich von
        // null (inklusive) bis zur Größe der Liste (exklusiv) liegen.
        int index = zufallsgenerator.nextInt(antworten.size());
        return antworten.get(index);
    }

    /**
     * Generiere eine Liste von Standardantworten, aus denen wir eine
     * auswählen können, wenn wir keine bessere Antwort wissen.
     */
    private void antwortlisteFuellen()
    {
        antworten.add("Das klingt seltsam. Können Sie das ausführlicher beschreiben?");
        antworten.add("Bisher hat sich noch kein Kunde darüber beschwert. \n" +
            "Welche Systemkonfiguration haben Sie?");
        antworten.add("Da brauche ich etwas ausführlichere Angaben.");
        antworten.add("Haben Sie geprüft, ob Sie einen Konflikt mit einer DLL haben?");
        antworten.add("Das steht im Handbuch. Haben Sie das Handbuch gelesen?");
        antworten.add("Das klingt alles etwas Wischi-Waschi. Haben Sie einen Experten\n" +
            "in der Nähe, der das besser beschreiben kann?");
        antworten.add("Das ist kein Fehler, das ist eine Systemeigenschaft!");
        antworten.add("Könnten Sie es anders erklären?");
        antworten.add("Haben Sie versucht, die App auf Ihrem Handy auszuführen?");
        antworten.add("Ich habe gerade StackOverflow überprüft - das ist auch keine Lösung.");
    }
}
```

In dieser Version haben wir den Teil, der die Antwortliste befüllt, in eine eigene Methode **antwortlisteFuellen** ausgelagert, die im Konstruktor aufgerufen wird. Dies garantiert, dass die Antwortliste befüllt wird, sobald ein **Beantworter**-Objekt erzeugt wird, doch durch diese Trennung wird der Quelltext der Klasse lesbarer und übersichtlicher.

Der interessanteste Abschnitt in dieser Klasse ist die Methode `generiereAntwort`. Ohne die Kommentare sieht sie folgendermaßen aus:

```
public String generiereAntwort()
{
    int index = zufallsgenerator.nextInt(antworten.size());
    return antworten.get(index);
}
```

Die erste Zeile in dieser Methode erledigt dreierlei:

- Sie lässt sich über den Aufruf von `size` die Größe der Antwortliste geben.
- Sie generiert eine Zufallszahl zwischen 0 (inklusive) und der Größe (exklusiv).
- Sie speichert die generierte Zufallszahl in der lokalen Variablen `index`.

Wenn Ihnen dies zu viel in einer Zeile ist, können Sie stattdessen auch schreiben:

```
int listengroesse = antworten.size();
int index = zufallsgenerator.nextInt(listengroesse);
```

Diese Anweisungen sind äquivalent zur ersten Zeile oben. Welche Version Sie bevorzugen, hängt davon ab, welche Sie besser lesbar finden.

Beachten Sie, dass dieser Abschnitt Zufallszahlen aus dem Bereich von 0 bis `listengroesse-1` (inklusive) erzeugt. Dies passt wunderbar zu den gültigen Indizes für eine `ArrayList`. Sie erinnern sich, dass der Wertebereich für Indizes einer `ArrayList` mit der Größe `listengroesse` von 0 bis `listengroesse-1` reicht. Die erzeugte Zufallszahl liefert uns somit einen perfekt geeigneten Index, um zufällig auf einen Eintrag in der Liste zugreifen zu können.

Die letzte Zeile der Methode lautet:

```
return antworten.get(index);
```

In dieser Zeile geschehen zwei Dinge:

- Sie fragt die Antwort an Position `index` mit der `get`-Methode ab.
- Sie liefert die gewählte Zeichenkette als Ergebnis der Methode mit der `return`-Anweisung zurück.

Wenn Sie nicht vorsichtig sind, kann es leicht passieren, dass Ihr Programm Zufallszahlen generiert, die außerhalb des gültigen Indexbereichs einer `ArrayList` liegen. Wenn Sie mit einem solchen ungültigen Index versuchen, auf ein Element zuzugreifen, erhalten Sie eine `IndexOutOfBoundsException`.

6.4.4 Die Dokumentation generischer Klassen

Bisher haben wir Sie aufgefordert, sich die Dokumentation der Klasse `String` aus dem Paket `java.lang` sowie die der Klasse `Random` aus dem Paket `java.util` anzusehen. Sie haben dabei möglicherweise bemerkt, dass einige Klassennamen in der Dokumentation etwas anders aussehen, wie beispielsweise `ArrayList<E>` oder `HashMap<K,V>`. Hinter den Klassennamen stehen hier Zusatzinformationen in spitzen Klammern. Solche Klassen heißen *parametrisierte Klassen* oder *generische Klassen*. Die Angaben in den spitzen Klammern sagen uns, welche zusätzlichen Typnamen wir zur Vervollständigung angeben müssen, um diese Klassen zu benutzen.

Wir haben dies bereits in Kapitel 4 angewendet, als wir die Klasse **ArrayList** benutzten, indem wir sie mit Typnamen wie **String** parametrisiert haben:

```
private ArrayList<String> notizen;  
private ArrayList<Posten> studenten;
```

Weil wir eine **ArrayList** mit einem beliebigen Klassentyp parametrisieren können, ist dies entsprechend in der Dokumentation der Java API angegeben. Wenn Sie sich beispielsweise die Methodenliste der Klasse **ArrayList<E>** ansehen, entdecken Sie Methoden wie:

```
boolean add(E o)  
E get(int index)
```

Hier sehen wir, dass der Typ der Objekte, die wir in eine **ArrayList** mit **add** einfügen dürfen, von dem Typ abhängt, mit dem die Klasse parametrisiert wurde. Das Gleiche gilt für den Ergebnistyp der Methode **get**. Im Endeffekt sagt die Dokumentation, dass wir, wenn wir ein Objekt des Typs **ArrayList<String>** erzeugen, ein Objekt mit den folgenden beiden Methoden erhalten:

```
boolean add(String o)  
String get(int index)
```

Wenn wir hingegen ein Objekt vom Typ **ArrayList<Student>** erzeugen, dann hat es diese beiden Methoden:

```
boolean add(Student o)  
Posten get(int index)
```

Wir werden Sie später in diesem Kapitel noch auffordern, sich die Dokumentation weiterer parametrisierter Typen anzusehen.

6.5 Pakete und Importe

Es gibt noch zwei Zeilen zu Anfang der Quelltextdatei, die wir hier ansprechen müssen:

```
import java.util.ArrayList;  
import java.util.Random;
```

Wir haben die **import**-Anweisung zum ersten Mal in Kapitel 4 gesehen. Wir werden sie nun etwas genauer untersuchen.

Die Java-Klassen in einer Klassenbibliothek stehen nicht automatisch zur Verfügung, so wie die übrigen Klassen in unserem aktuellen Projekt. Stattdessen müssen wir in unserem Quelltext deklarieren, dass wir eine Klasse aus der Bibliothek benutzen möchten. Dies wird *Importieren* der Klasse genannt und geschieht entsprechend mit einer **import**-Anweisung. Die **import**-Anweisung hat die Form

```
import qualifizierter-Klassenname;
```

Da die Java-Bibliothek aus mehreren Tausend Klassen besteht, ist ein Strukturierungsmechanismus notwendig, mit dem der Umgang mit dieser Menge erleichtert wird. Java benutzt Pakete (*package*), um Bibliotheksklassen in zusammen-

gehörigen Gruppen zu bündeln. Pakete können geschachtelt werden, d.h., ein Paket kann weitere Pakete enthalten.

Die Klassen `ArrayList` und `Random` gehören beide zum Paket `java.util`. Diese Information steht in der Klassendokumentation. Der vollständige oder qualifizierte Name einer Klasse besteht aus dem Namen seines Pakets, gefolgt von einem Punkt und dem Klassennamen. Die qualifizierten Namen der beiden genannten Klassen sind somit `java.util.ArrayList` und `java.util.Random`.

Java erlaubt auch, dass ganze Pakete mit Anweisungen der Form

```
import Paketname.*;
```

importiert werden. Die folgende Anweisung würde somit alle Klassennamen aus dem Paket `java.util` importieren:

```
import java.util.*;
```

Die Klassen einzeln aufzuführen, so wie in unserer ersten Version, bedeutet etwas mehr Schreibarbeit, dokumentiert aber auch deutlicher, welche Klassen tatsächlich benutzt werden. Deshalb werden wir überwiegend dem Stil des ersten Beispiels folgen, also jede Klasse einzeln importieren.

Es gibt eine Ausnahme für die Importregeln: Einige Klassen werden so oft benötigt, dass sie praktisch in jede Klasse importiert werden müssten. Diese Klassen sind im Paket `java.lang` definiert und dieses Paket wird automatisch in jede Klasse importiert. `String` ist ein Beispiel für eine Klasse aus `java.lang`.

Übung 6.22 Implementieren Sie die hier diskutierte Lösung mit den zufälligen Antworten in Ihrer Version des Projekts *Technischer-Kundendienst*.

Übung 6.23 Was passiert, wenn Sie weitere Antworten in die Antwortliste einfügen (oder einige entfernen)? Wird die zufällige Auswahl einer Antwort weiterhin funktionieren? Warum oder warum nicht?

Die Lösung für diese Aufgaben finden Sie auch im Begleitmaterial unter dem Namen *Technischer-Kundendienst2*. Wir empfehlen jedoch, dass Sie diese Lösung selbst als eine Erweiterung der Basisversion implementieren.

6.6 Benutzung von Map-Klassen für Abbildungen

Wir haben nun eine Lösung für unser Kundendienstsystem, die zufällige Antworten generiert. Das ist besser als unsere erste Version, aber noch nicht wirklich überzeugend. Insbesondere werden die Antworten nach wie vor nicht von den Eingaben des Benutzers beeinflusst. Das wollen wir nun verbessern.

Die Idee ist, dass wir eine Menge von Wörtern definieren, die typischerweise in Anfragen auftauchen, und diese Wörter mit entsprechenden Antworten ver-

knüpfen. Wenn die Eingabe des Benutzers eines der uns bekannten Wörter enthält, dann können wir eine passende Antwort geben. Das ist immer noch eine sehr ungenaue Methode, da sie weder auf den Inhalt einer Anfrage eingeht noch den Kontext mit einbezieht. Sie kann dennoch überraschend effektiv sein und es ist ein guter nächster Schritt.

Für diesen Schritt werden wir eine **HashMap** verwenden. Sie finden die Dokumentation der Klasse **HashMap** in der Java-Bibliothek. **HashMap** ist eine Spezialisierung von **Map**, die ebenfalls dokumentiert ist. Sie werden sehen, dass Sie beide lesen müssen, um zu verstehen, was eine **HashMap** ist und wie sie funktioniert.

Übung 6.24 Was ist eine **HashMap**? Welchen Zweck hat sie und wie benutzt man sie?

Beantworten Sie diese Fragen schriftlich. Benutzen Sie die Dokumentation von **Map** und **HashMap** in der Java-Bibliothek, um diese Fragen zu beantworten. Beachten Sie, dass es schwierig sein kann, alles zu verstehen, da die Dokumentation dieser Klassen nicht sehr gut ist. Wir werden die Details in diesem Kapitel noch besprechen, aber bevor Sie weiterlesen, sollten Sie versuchen, möglichst viel auf eigene Faust herauszufinden.

Übung 6.25 **HashMap** ist eine parametrisierte Klasse. Benennen Sie die Methoden, die von den Typen abhängen, mit denen die Klasse parametrisiert wird. Denken Sie, dass derselbe Typ für beide Typparameter benutzt werden kann?

6.6.1 Das Konzept einer Map

Konzept

Eine **Map** ist eine Sammlung, die Schlüssel-Wert-Paare als Einträge enthält. Ein Wert kann ausgelesen werden, indem ein Schlüssel angegeben wird.

Eine Map ist eine Sammlung von Schlüssel-Wert-Paaren, wobei sowohl Schlüssel als auch Werte Objekte sind. Wie auch eine **ArrayList** kann eine **Map** eine beliebige Anzahl von Einträgen haben. Ein Unterschied zwischen einer **ArrayList** und einer **Map** ist, dass bei einer **Map** ein Eintrag nicht ein einzelnes Objekt ist, sondern ein Paar von Objekten. Ein solches Paar besteht aus einem *Schlüssel*-Objekt und einem *Wert*-Objekt.

Bei einer Map verwenden wir nicht einen Index, um ein Element einer Sammlung zu referenzieren (wie wir es bei einer **ArrayList** tun), sondern wir nehmen das Schlüssel-Objekt, um das Wert-Objekt zu bekommen. Im Deutschen verwenden wir den Begriff *Abbildung* für dieses Konzept: Ein Schlüssel wird auf einen Wert abgebildet.

Ein Alltagsbeispiel für eine solche Abbildung ist eine Kontaktliste. Eine Kontaktliste enthält Einträge, die Paare aus einem Namen und einer Telefonnummer sind. Wir verwenden eine Kontaktliste, indem wir einen Namen nachschlagen und dann die zugeordnete Nummer verwenden. Wir benutzen keinen Index – also die Position eines Eintrags in der Kontaktliste –, um sie zu finden.

Eine **Map** kann so strukturiert sein, dass das Nachschlagen eines Wertes für einen Schlüssel einfach sein kann. Bei einer Kontaktliste beispielsweise ist dies durch

die alphabetische Ordnung gewährleistet. Da die Einträge alphabetisch nach ihren Schlüsseln sortiert sind, ist das Finden eines Schlüssels und damit des zugeordneten Werts einfach. Die umgekehrte Suche (einen Schlüssel für einen Wert suchen, also einen Namen zu einer Telefonnummer) ist mit einer solchen Abbildung nicht so einfach. Wie im Falle einer Kontaktliste wäre die umgekehrte Suche zwar möglich, aber im Vergleich extrem zeitaufwendig. Abbildungen sind somit ideal für eine einseitig gerichtete Suche, wenn wir den Suchschlüssel kennen und den zugeordneten Wert benötigen.

6.6.2 Die Benutzung einer HashMap

HashMap ist eine spezifische Implementierung einer **Map**. Die wichtigsten Methoden einer **HashMap** sind **put** und **get**.

Die Methode **put** fügt der Abbildung einen Eintrag hinzu und **get** liefert für einen gegebenen Schlüssel einen Wert. Der folgende Quelltextabschnitt erzeugt eine **HashMap** und fügt drei Einträge ein. Jeder Eintrag ist ein Schlüssel-Wert-Paar, bestehend aus einem Namen und einer Telefonnummer.

```
HashMap<String, String> kontakte = new HashMap<>();
kontakte.put("Günter Schmidt", "(0531) 392 4587");
kontakte.put("Harald Lorant", "(089) 7386 6632");
kontakte.put("Werner Jauch", "(030) 4532 7761");
```

Wie auch bei einer **ArrayList** müssen wir, wenn wir eine **HashMap**-Variable deklarieren und ein **HashMap**-Objekt erzeugen, den Typ der Objekte angeben, die in der **Map** enthalten sein sollen; zusätzlich müssen wir hier auch den Typ des Schlüssels angeben. Für unsere Kontaktliste verwenden wir **String** sowohl für den Schlüsseltyp als auch für den Elementtyp, aber die beiden Typen müssen im Allgemeinen nicht gleich sein.

Wie wir bereits in Abschnitt 4.4.2 gesehen haben, müssen wir bei der Erzeugung von Objekten generischer Klassen und deren Zuweisung an Variablen die generischen Typen (hier **<String, String>**) nur einmal auf der linken Seite der Zuweisung angeben und können den Diamant-Operator in der Objekterzeugung auf der rechten Seite verwenden. Die generischen Typen für die Objekterzeugung werden dann von der Variablendeklaration kopiert.

Die folgenden Anweisungen suchen die Telefonnummer von Harald Lorant und geben sie aus.

```
String nummer = kontakte.get("Harald Lorant");
System.out.println(nummer);
```

Beachten Sie, dass hier der Schlüssel (der Name „Harald Lorant“) an die **get**-Methode übergeben wird, um den Wert (die Telefonnummer) zu bekommen.

Lesen Sie die Dokumentation von **get** und **put** der Klasse **HashMap** erneut, um festzustellen, ob diese Erklärungen Ihrem aktuellen Verständnis entsprechen.

Übung 6.26 Wie überprüfen Sie, wie viele Einträge in einer Abbildung vorhanden sind?

Übung 6.27 Legen Sie eine Klasse **MapTester** an (entweder in Ihrem aktuellen Projekt oder in einem eigenen). Verwenden Sie in diesem Projekt eine **HashMap**, um eine Kontaktliste wie oben dargestellt zu implementieren. (Vergessen Sie nicht, **java.util.HashMap** zu importieren.) Implementieren Sie zwei Methoden in dieser Klasse:

```
public void nummerEintragen(String name, String nummer)
```

und

```
public String nummerSuchen(String name)
```

Diese Methoden sollten die Methoden **put** und **get** der Klasse **HashMap** verwenden, um ihre Funktionalität zu implementieren.

Übung 6.28 Was passiert, wenn Sie bei einer solchen Abbildung einen Eintrag mit einem Schlüssel vornehmen, der bereits eingetragen ist?

Übung 6.29 Was passiert, wenn Sie bei einer solchen Abbildung zwei Einträge mit demselben Wert, aber mit verschiedenen Schlüsseln vornehmen?

Übung 6.30 Wie prüfen Sie, ob ein gegebener Schlüssel bereits in einer Abbildung eingetragen ist? (Geben Sie einen Java-Quelltext an.)

Übung 6.31 Was passiert, wenn Sie einen Wert suchen und der Schlüssel in der Abbildung nicht existiert?

Übung 6.32 Wie geben Sie alle Schlüssel aus, die aktuell in einer Abbildung gespeichert sind?

6.6.3 Benutzung einer Abbildung für das Kundendienstsystem

Für unser Kundendienstsystem können wir eine solche Abbildung gut gebrauchen, indem wir bekannte Wörter als Schlüssel nehmen und zugeordnete Antworten als Werte. Listing 6.4 zeigt ein Beispiel, in dem eine **HashMap** namens **antwortMap** erzeugt wird und drei Einträge vorgenommen werden. Beispielsweise ist das Wort „langsam“ mit folgendem Text verknüpft:

Ich vermute, dass das mit Ihrer Hardware zu tun hat. Ein Upgrade für Ihren Prozessor sollte diese Probleme lösen. Haben Sie ein Problem mit unserer Software?

Wenn nun jemand eine Anfrage stellt, die das Wort „langsam“ enthält, können wir diese Antwort suchen und ausgeben. Beachten Sie, dass die Antwort-Zeichenkette im Quelltext über mehrere Zeilen geht, die einzelnen Teile aber mit **+** verknüpft sind und deshalb als ein einzelner Wert in die **HashMap** eingetragen werden.

```

private HashMap<String, String> antwortMap;

...

public Beantworter()
{
    antwortMap = new HashMap<>();
    antwortMapBefuellen();
}

/**
 * Trage alle bekannten Stichwörter mit ihren verknüpften
 * Antworten in die Map 'antwortMap' ein.
 */
private void antwortMapBefuellen()
{
    antwortMap.put("langsam",
        "Ich vermute, dass das mit Ihrer Hardware zu tun hat. Ein Upgrade\n" +
        "für Ihren Prozessor sollte diese Probleme lösen.\n" +
        "Haben Sie ein Problem mit unserer Software?");
    antwortMap.put("fehler",
        "Wissen Sie, jede Software hat Fehler. Aber unsere Entwickler\n" +
        "arbeiten sehr hart daran, diese Fehler zu beheben.\n" +
        "Können Sie das Problem ein wenig genauer beschreiben?");
    antwortMap.put("teuer",
        "Unsere Preise sind absolute Marktpreise. Haben Sie sich mal umgesehen\n" +
        "und wirklich unser Leistungsspektrum verglichen?");
}

```

Listing 6.4

Ausgewählte Stichwörter mit Antworten verknüpfen.

Ein erster Versuch, eine Methode zum Erzeugen der Antworten zu schreiben, könnte so aussehen wie die Methode **generiereAntwort** unten. Zur Vereinfachung nehmen wir hier vorläufig an, dass der Benutzer lediglich ein einzelnes Wort (etwa „langsam“) eingegeben hat.

```

public String generiereAntwort(String wort)
{
    String antwort = antwortMap.get(wort);
    if(antwort != null) {
        return antwort;
    }
    else {
        // Wenn wir hierher gelangen, wurde das Stichwort nicht erkannt.
        // In diesem Fall wählen wir eine unserer Standardantworten.
        return standardantwortAuswählen();
    }
}

```

In diesem Quelltextabschnitt schlagen wir ein Wort, das der Benutzer eingegeben hat, in der Map nach. Wenn wir dort einen Eintrag finden, liefern wir die zugeordnete Antwort. Wenn wir keinen Eintrag für das Wort finden, rufen wir die Methode **standardantwortAuswählen** auf. Diese Methode kann die Anweisungen aus unserer vorherigen Version von **generiereAntwort** enthalten, die zufällig eine der Standardantworten auswählt (wie in Listing 6.3 dargestellt). Die neue Verarbeitungslogik ist somit, dass wir eine angemessene Antwort liefern, wenn wir ein Stichwort erkennen, ansonsten wählen wir zufällig eine unserer Standardantworten aus.

Übung 6.33 Implementieren Sie die hier diskutierten Veränderungen in Ihrer Version des Kundendienstsystems. Testen Sie es, um ein Gefühl für die Verbesserung zu bekommen.

Dieser Ansatz, Stichwörter mit Antworten zu verknüpfen, funktioniert nur, solange der Benutzer statt ganzer Sätze nur einzelne Wörter eingibt. Die letzte Verbesserung unserer Anwendung wird sein, dass ein Benutzer wieder ganze Sätze eingeben kann und wir dann eine passende Antwort auswählen, wenn wir eines der Wörter in der Anfrage erkennen.

Dies stellt uns vor das Problem, dass wir Stichwörter in dem Satz erkennen müssen, den der Benutzer eingibt. In der momentanen Version wird die Benutzereingabe durch den **Eingabeleser** als eine einzelne Zeichenkette geliefert. Wir werden dies nun so abändern, dass der **Eingabeleser** eine Menge von Wörtern liefert. Technisch wird das eine Menge von Zeichenketten sein, von denen jede ein Wort aus der Benutzereingabe repräsentiert.

Wenn wir das hinbekommen, dann können wir diese Wortmenge an den **Beantworter** weiterreichen, der dann jedes Wort in der Menge daraufhin überprüft, ob es bekannt ist und mit einer Antwort verknüpft ist.

Um dies in Java umzusetzen, benötigen wir zweierlei: Wir müssen wissen, wie wir eine Zeichenkette mit einem ganzen Satz in einzelne Wörter zerlegen können, und wir müssen mit Mengen umgehen können. Diese Punkte werden in den nächsten beiden Abschnitten behandelt.

6.7 Der Umgang mit Mengen

Konzept

Eine **Menge (Set)** ist eine Sammlung, in der jedes Element nur maximal einmal enthalten ist. Die Elemente einer Menge haben keine spezifische Ordnung.

Die Java-Bibliothek bietet verschiedene Varianten von Mengen (**set**), die durch unterschiedliche Klassen implementiert sind. Wir werden im Folgenden ein **HashSet** benutzen.

Übung 6.34 Was sind die Gemeinsamkeiten und die Unterschiede zwischen einem **HashSet** und einer **ArrayList**? Benutzen Sie die Beschreibungen von **Set**, **HashSet**, **List** und **ArrayList** in der Dokumentation der Bibliothek für Ihre Untersuchung, da ein **HashSet** ein spezieller Fall eines **Set** ist und eine **ArrayList** ein spezieller Fall einer **List**.

Die beiden Dienstleistungen, die wir von einer Menge erwarten, sind das Eintragen von Elementen und das spätere Abfragen dieser Elemente. Glücklicherweise sind uns diese Anforderungen nicht neu. Betrachten Sie folgenden Quelltextabschnitt:

```
import java.util.HashSet;
import java.util.Iterator;
...
HashSet<String> meineMenge = new HashSet<>();
meineMenge.add("eins");
meineMenge.add("zwei");
meineMenge.add("drei");
```

Vergleichen Sie diese Anweisungen mit denen, die für das Einfügen der Elemente in eine **ArrayList** notwendig sind. Es gibt praktisch keinen Unterschied, außer dass wir diesmal ein **HashSet** erzeugen statt einer **ArrayList**. Nun wollen wir das Iterieren über die Menge betrachten:

```
for (String eintrag : meineMenge) {
    etwas mit diesem Eintrag tun
}
```

Auch hier sehen wir, dass dies die gleichen Anweisungen sind, die wir in Kapitel 4 für das Iterieren über eine **ArrayList** benutzt haben.

Kurz gesagt: Der Umgang mit den Sammlungen in Java ist für die verschiedenen Sammlungstypen sehr ähnlich. Sobald Sie den Umgang mit einer verstanden haben, können Sie alle benutzen. Die tatsächlichen Unterschiede liegen im Verhalten der jeweiligen Sammlung. Eine Liste (**List**) beispielsweise hält ihre Elemente in der gewünschten Reihenfolge, bietet den Zugriff über einen Index und kann dasselbe Element mehrfach enthalten. Eine Menge (**Set**) hingegen definiert für ihre Elemente keine Reihenfolge (die **for-each**-Schleife kann die Elemente in einer anderen Reihenfolge liefern als die, in der sie eingefügt wurden) und garantiert, dass jedes Element nur maximal einmal in der Menge vorkommt. Wenn ein Element ein zweites Mal eingefügt wird, hat dies schlicht keinen Effekt.

6.8 Zeichenketten zerlegen

Nachdem wir den Umgang mit Mengen betrachtet haben, können wir nun untersuchen, wie wir eine Eingabezeile in mehrere Wörter zerlegen und diese in einer Menge ablegen können. Die Lösung ist als eine Version der Methode **gibEingabe** der Klasse **Eingabeleser** realisiert (Listing 6.5).

```
/**
 * Lies eine Zeile von der Konsole und liefere sie als eine
 * Menge von Wörtern zurück.
 *
 * @return eine Menge von Zeichenketten, von denen jede ein
 *         vom Benutzer getipptes Wort repräsentiert
 */
public HashSet<String> gibEingabe()
{
    System.out.print("> "); // Eingabebereitschaft anzeigen
    String eingabezeile = scanner.nextLine().trim().toLowerCase();

    String[] wortArray = eingabezeile.split(" "); //Trennung bei Leerzeichen

    // Wörter aus dem Array in das HashSet einfügen
    HashSet<String> woerter = new HashSet<>();
    for(String wort : wortArray) {
        woerter.add(wort);
    }

    return woerter;
}
```

Listing 6.5

Die Methode **gibEingabe** liefert eine Menge von Wörtern.

Hier verwenden wir zusätzlich zur Klasse **HashSet** die Methode **split** der Klasse **String**, die ebenfalls in der Java-Standardbibliothek definiert ist.

Die Methode `split` kann einen String in getrennte Substrings aufteilen und diese in einem String-Array zurückliefern (Arrays werden im nächsten Kapitel noch ausführlich besprochen). Der Parameter der Methode `split` definiert, an welcher Art von Zeichen der Original-String zerlegt werden soll. Wir haben definiert, dass wir unseren String an jedem Leerzeichen zerlegen wollen.

Die nächsten Zeilen erzeugen ein `HashSet` und kopieren die Wörter aus dem Array in dieses hinein, bevor das Set zurückgeliefert wird.³

Übung 6.35 Die Methode `split` ist mächtiger, als sie in diesem Beispiel auf den ersten Blick aussieht. Wie können Sie exakt festlegen, wie ein String zerlegt werden soll? Geben Sie einige Beispiele an.

Übung 6.36 Wie würden Sie die Methode `split` aufrufen, wenn Sie einen String an Leerzeichen oder Tabulatorzeichen zerlegen wollen? Wie können Sie einen String zerlegen, in dem die Wörter durch Doppelpunkte (:) getrennt sind?

Übung 6.37 Worin unterscheidet sich das Ergebnis, wenn die Wörter statt in einem `HashSet` in einer `ArrayList` zurückgeliefert werden?

Übung 6.38 Was passiert, wenn mehr als ein Leerzeichen zwischen zwei Wörtern steht (beispielsweise zwei oder drei Leerzeichen)? Gibt es ein Problem?

Übung 6.39 *Zusatzaufgabe.* Lesen Sie die Fußnote über die Methode `Arrays.asList`. Finden und lesen Sie die späteren Abschnitte über Klassenvariablen und Klassenmethoden. Erklären Sie in Ihren eigenen Worten, wie diese Lösung funktioniert.

Übung 6.40 Welche weiteren Methoden bietet die Klasse `Arrays`?

Übung 6.41 Erzeugen Sie eine Klasse namens `SortierenTest`. Erzeugen Sie in dieser Klasse eine Methode, die ein Array von `int`-Werten als Parameter entgegennimmt und die Elemente sortiert (kleinstes Element zuerst) auf der Konsole ausgibt.

6.9 Abschluss des Kundendienstsystems

Um alles zusammensetzen zu können, müssen wir nun auch noch die Klassen `Kundendienstsystem` und `Beantworter` so anpassen, dass sie mit einer Menge von Worten statt mit einer einzelnen Zeichenkette umgehen können. Listing 6.6 zeigt

³ Es gibt einen kürzeren und noch eleganteren Weg. Sie können schreiben: `HashSet<String> woerter = new HashSet<>(Arrays.asList(wortArray));` um alle vier Zeilen Quelltext zu ersetzen. Diese Lösung benutzt die Klasse `Arrays` aus der Standardbibliothek und eine *statische Methode* (häufig auch *Klassenmethode* genannt), die wir hier noch nicht diskutieren wollen. Wenn Sie neugierig sind, dann lesen Sie den Abschnitt 6.16 über Klassenmethoden und probieren diese Version aus.

die neue Version der Methode **starten** aus der Klasse **Kundendienstsystem**. Sie hat sich nicht sehr stark verändert. Die Änderungen sind im Einzelnen:

- Die Variable **eingabe** für das Ergebnis des Aufrufs **leser.gibEingabe()** ist nun vom Typ **HashSet**.
- Die Prüfung auf Beendigung der Anwendung wird mit der Methode **contains** der Klasse **HashSet** vorgenommen statt mit einer **String**-Methode. (Lesen Sie diese Methode in der Dokumentation nach.)
- Die Klasse **HashSet** muss mit einer **import**-Anweisung importiert werden (im Listing nicht gezeigt).

```
public void starten()
{
    boolean fertig = false;

    willkommenstextAusgeben();

    while(!fertig) {
        HashSet<String> eingabe = leser.gibEingabe();

        if(eingabe.contains("ade")) {
            fertig = true;
        }
        else {
            String antwort = beantworter.generiereAntwort(eingabe);
            System.out.println(antwort);
        }
    }
    abschiedstextAusgeben();
}
```

Listing 6.6

Die endgültige Version der Methode **starten**.

Schließlich müssen wir noch die Methode **generiereAntwort** in der Klasse **Beantworter** so erweitern, dass sie eine Menge von Wörtern als Parameter akzeptiert. Sie muss dann über diese Wörter iterieren und bei jedem überprüfen, ob es in der Map mit den bekannten Stichwörtern enthalten ist. Wenn eines der Wörter erkannt wird, liefern wir sofort die entsprechende Antwort zurück. Wenn wir keines der Wörter erkennen, liefern wir wie vorher eine der Standardantworten. Listing 6.7 zeigt diese Lösung.

```
public String generiereAntwort(HashSet<String> woerter)
{
    for (String wort : woerter) {
        String antwort = antwortMap.get(wort);
        if(antwort != null) {
            return antwort;
        }
    }

    // Wenn wir hierher gelangen, wurde keines der Eingabewörter erkannt.
    // In diesem Fall wählen wir eine unserer Standardantworten (die
    // wir geben, wenn uns nichts Besseres mehr einfällt).

    return standardantwortAuswählen();
}
```

Listing 6.7

Die endgültige Version der Methode **generiereAntwort**.

Dies ist die letzte Änderung an dieser Anwendung, die wir in diesem Kapitel diskutieren. Die Lösung im Projekt *Technischer-Kundendienst-komplett* enthält all diese Änderungen. Sie enthält auch eine größere Anzahl Verknüpfungen von Stichwörtern zu Antworten, als in diesem Kapitel gezeigt wird.

Es sind noch viele Verbesserungen dieser Anwendung denkbar. Wir werden diese hier nicht diskutieren. Stattdessen geben wir Verbesserungsvorschläge in den Übungen, die der Leser vornehmen kann. Einige davon sind recht anspruchsvolle Programmieraufgaben.

Übung 6.42 Implementieren Sie die oben diskutierten Änderungen in Ihrer Version des Projekts.

Übung 6.43 Fügen Sie Ihrer Anwendung weitere Wort-Antwort-Paare hinzu. Einige können Sie aus der gegebenen Lösung kopieren und einige selbst hinzufügen.

Übung 6.44 Stellen Sie sicher, dass die gleiche Standardantwort nicht zweimal hintereinander ausgewählt wird.

Übung 6.45 Einige Wörter werden auf dieselbe Antwort abgebildet. Erweitern Sie Ihr System so, dass es mit Synonymen und abgeleiteten Wörtern explizit umgeht, damit Sie nicht mehrere Einträge in der Map für dieselbe Antwort vornehmen müssen.

Übung 6.46 Identifizieren Sie auch mehrere passende Wörter in der Benutzereingabe und reagieren Sie mit einer passenderen Antwort.

Übung 6.47 Wenn kein Wort erkannt wird, können Sie auch auf andere Weise auf Wörter aus der Eingabe reagieren, beispielsweise mit Standardantworten auf Fragen mit „Wie“, „Warum“ und „Wer“.

6.10 Autoboxing und Wrapper-Klassen

Wir haben gesehen, dass die Sammlungsklassen (wie **ArrayList**) mit der geeigneten Parametrisierung Objekte jedes Objekttyps speichern können. Es bleibt jedoch ein Problem: Java hat auch einige Typen, die keine Objekttypen sind.

Wir erinnern uns, dass die primitiven Typen wie **int**, **boolean**, **char** etc. sich grundsätzlich von Objekttypen unterscheiden. Ihre Werte sind nicht Instanzen von Klassen und können normalerweise nicht in Sammlungen eingetragen werden.

Das ist schade. Es sind Situationen denkbar, in denen wir beispielsweise Listen von **int**-Werten oder Mengen von Zeichen (**char**) benötigen. Was können wir tun?

Die Lösung von Java für dieses Problem sind die sogenannten *Wrapper-Klassen* (vom Englischen *to wrap*: einpacken, umwickeln). Für jeden primitiven Typ gibt es eine korrespondierende Wrapper-Klasse, die den zugeordneten primitiven Typ repräsentiert, aber ein echter Objekttyp ist. Die Wrapper-Klasse für **int** ist beispielsweise **Integer**. Eine komplette Liste der primitiven Typen und ihrer Wrapper-Klassen findet sich in Anhang B.

Die folgende Anweisung „wickelt“ den Wert der Variablen **ix** vom primitiven Typ **int** in ein **Integer**-Objekt:

```
Integer iwrap = new Integer(ix);
```


Und nun kann `wrap` beispielsweise in einer Sammlung vom Typ `ArrayList<Integer>` gespeichert werden. Das Speichern von primitiven Werten in Objektsammlungen wird jedoch noch mehr durch ein Sprachkonstrukt erleichtert, das als *Autoboxing* (übersetzbar mit „automatischer Verpackung“) bezeichnet wird.

Immer dann, wenn der Wert eines primitiven Typs in einem Zusammenhang verwendet wird, der einen Wrapper-Typ erfordert, verpackt der Compiler den Wert des primitiven Typs automatisch in ein Objekt der passenden Wrapper-Klasse. Dies bedeutet, dass die Werte primitiver Typen direkt in eine Sammlung eingefügt werden können.

```
private ArrayList<Integer> notenListe;
...
public void speichereNoteInListe(int note)
{
    notenListe.add(note);
}
```

Die inverse Operation – das *Unboxing* – wird ebenfalls automatisch ausgeführt, sodass das Auslesen aus einer Sammlung etwa folgendermaßen aussehen könnte:

```
int ersteNote = notenListe.remove(0);
```

Autoboxing wird ebenfalls angewendet, wenn der Wert eines primitiven Typs als Parameter an eine Methode übergeben wird, die einen Wrapper-Typ erwartet, und wenn der Wert eines primitiven Typs in einer Variablen eines Wrapper-Typs gespeichert wird. In analoger Weise wird Unboxing angewendet, wenn der Wert eines Wrapper-Typs als Parameter an eine Methode übergeben wird, die einen primitiven Typ erwartet, und wenn er in einer Variablen mit einem primitiven Typ gespeichert wird. Beachten Sie, dass diese Verfahrensweise dazu führt, dass es fast so scheint, als ob man primitive Typen in Sammlungen speichern könnte. Doch der Typ der Sammlung muss immer noch unter Verwendung des Wrapper-Typs (z.B. `ArrayList<Integer>` und nicht `ArrayList<int>`) deklariert werden.

Konzept

Autoboxing

Autoboxing wird automatisch durchgeführt, wenn der Wert eines primitiven Typs in einem Kontext verwendet wird, in dem ein Wrapper-Typ erwartet wird.

6.10.1 Verwalten der Nutzungszählung

Die Kombination einer Map mit Autoboxing bietet eine einfache Möglichkeit, die Nutzungszählung von Objekten zu verwalten. Wir nehmen beispielsweise an, dass das Unternehmen, welches das System *Technischer-Kundendienst* verwendet, Beschwerden von seinen Kunden bekommt, weil einige der Antworten keinen Bezug zu den gestellten Fragen haben. Das Unternehmen könnte dann entscheiden, die in den Fragen verwendeten Wörter zu analysieren sowie weitere gezielte Antworten für diejenigen Wörter hinzuzufügen, die am häufigsten vorkommen und für die es keine fertigen Antworten gibt. Listing 6.8 zeigt einen Teil der Klasse `Wortzaehler`, die im Projekt *Technischer-Kundendienst-Analyse* zu finden ist.

Listing 6.8

Die Klasse **Wortzaehler** wird eingesetzt, um Worthäufigkeiten zu zählen.

```
import java.util.HashMap;
import java.util.HashSet;

/**
 * Aufzeichnungen, wie häufig jedes Wort vom Benutzer
 * eingegeben wurde.
 *
 * @author Michael Kölling und David J. Barnes
 * @version 1.0 (2016.02.29)
 */
public class Wortzaehler
{
    // Jedes Wort wird mit einer Zählung verbunden.
    private HashMap<String, Integer> zaehlungen;

    /**
     * Erzeuge einen Wortzähler.
     */
    public Wortzaehler()
    {
        zaehlungen = new HashMap<>();
    }

    /**
     * Aktualisiere die Zählungen aller Wörter in der Eingabe.
     * @param eingabe eine Menge von Wörtern, die vom Benutzer eingegeben wurden
     */
    public void woerterHinzufuegen(HashSet<String> eingabe)
    {
        for(String wort : eingabe) {
            int zaehler = zaehlungen.getOrDefault(wort, 0);
            zaehlungen.put(wort, zaehler + 1);
        }
    }
}
```

Die Methode **woerterHinzufuegen** erhält dieselbe Wortmenge, die dem **Beantworter** übergeben wird, sodass jedem Wort eine Häufigkeit zugeordnet werden kann. Die Häufigkeiten werden in einer Map gespeichert, die Zeichenketten auf ganzen Zahlen abbildet.

Beachten Sie den Einsatz der **HashMap**-Methode **getOrDefault**, die zwei Parameter besitzt: einen Schlüssel und einen vorgegebenen Wert. Wird der Schlüssel in der Map bereits verwendet, dann gibt diese Methode den entsprechenden Wert zurück. Falls aber der Schlüssel nicht verwendet wird, dann wird der vorgegebene Wert anstelle von **null** geliefert. Dadurch wird vermieden, zwei unterschiedliche Folgeaktionen zu schreiben, je nachdem, welcher Schlüssel in Benutzung ist und welcher nicht. Wenn wir stattdessen **get** verwenden würden, müssten wir so etwas wie Folgendes schreiben:

```
Integer zaehler = zaehlungen.get(wort);
if(zaehler == null) {
    zaehlungen.put(wort, 1);
}
else {
    zaehlungen.put(wort, zaehler + 1);
}
```

Beachten Sie, wie Autoboxing und Unboxing mehrere Male in diesen Beispielen eingesetzt werden.

Übung 6.48 Was macht die Methode `putIfAbsent` von `HashMap`?

Übung 6.49 Fügen Sie eine Methode zur Klasse `Wortzaehler` im Projekt *Technischer-Kundendienst-Analyse* hinzu, um die Häufigkeit jedes Wortes auszugeben, nachdem die „ade“-Nachricht ausgegeben wurde.

Übung 6.50 Geben Sie nur von denjenigen Wörtern die Häufigkeiten aus, die noch keine Schlüssel in `antwortMap` im der `Beantworter`-Klasse sind. Sie werden dazu eine sondierende Methode für `antwortMap` bereitstellen müssen.

6.11 Die Klassendokumentation schreiben

Während Sie an einem Projekt arbeiten, ist es wichtig, dass Sie neben der Entwicklung des Quelltextes auch die Dokumentation erstellen. Es ist leider nach wie vor noch allgemein üblich, dass Programmierer das Dokumentieren nicht sehr ernst nehmen, was später häufig zu ernsthaften Problemen führt.

Wenn Sie nicht genügend dokumentieren, dann kann es für einen anderen Programmierer (oder sogar für Sie selbst!) schwierig sein, Ihre Klassen zu verstehen. Typischerweise müssen Sie dann die Implementierung untersuchen, um Ihre Arbeitsweise zu verstehen. Das klappt eventuell noch in kleinen Studentenprojekten, aber in realen Projekten führt das zu erheblichen Schwierigkeiten.

Es ist bei kommerziellen Anwendungen durchaus üblich, dass der Quelltext aus mehreren Hunderttausend Zeilen in mehreren Tausend Klassen besteht. Stellen Sie sich vor, Sie müssten all das lesen, um die Arbeitsweise zu verstehen! Das kann eine einzelne Person nicht bewältigen.

Als wir Klassen wie `HashSet` oder `Random` aus der Java-Bibliothek benutzt haben, haben wir uns ausschließlich auf die Dokumentation verlassen, um ihre Dienstleistungen zu verstehen. Wir haben uns nicht ein einziges Mal die Implementierung dieser Klassen angesehen. Das hat funktioniert, weil diese Klassen einigermaßen gut dokumentiert sind (obwohl auch da noch Verbesserungen denkbar sind). Es wäre sehr viel schwieriger für uns geworden, wenn wir uns die Implementierungen der Klassen hätten ansehen müssen, bevor wir sie benutzen können.

In einem Entwicklungsteam für Software werden die Implementierungen der Klassen üblicherweise von mehreren Programmierern gemeinsam benutzt. Während Sie möglicherweise für die Klasse `Kundendienstsystem` aus unserem letzten Beispiel zuständig sind, implementiert ein anderer vielleicht die Klasse `Eingabeleser`. Es kann also vorkommen, dass Sie die eine Klasse implementieren und dabei Methoden von anderen Klassen aufrufen.

Das Argument, das bei Bibliotheksklassen so schlüssig ist, gilt in gleicher Weise für die Klassen, die Sie selbst erstellen: Wenn wir sie benutzen können, ohne dass wir ihre Implementierung lesen und verstehen müssen, dann wird uns das die Arbeit erleichtern. Wie bei den Bibliotheksklassen wollen wir nur die Schnittstelle einer Klasse sehen, nicht ihre Implementierung. Deshalb ist eine gute Klassendokumentation für Ihre eigenen Klassen ebenso wichtig.

Konzept

Die **Dokumentation** einer Klasse sollte genau die Informationen bieten, die andere Programmierer benötigen, um die Klasse ohne einen Blick auf ihre Implementierung benutzen zu können.

Java-Systeme bieten ein Werkzeug namens **javadoc** an, mit dem solche Schnittstellenbeschreibungen aus Quelltexten generiert werden können. Die Dokumentation der Standardbibliothek beispielsweise, die wir uns angesehen haben, ist aus den Quelltexten der Klassen mithilfe von **javadoc** erzeugt worden.

6.11.1 javadoc in BlueJ nutzen

Die BlueJ-Umgebung benutzt **javadoc**, um für Ihre Klassen die Dokumentation auf zwei Wegen zu erzeugen:

- Sie können die Dokumentation für eine einzelne Klasse betrachten, indem Sie im aufklappbaren Listenfeld rechts oben im Editorfenster vom Eintrag **QUELLTEXT** zum Eintrag **DOKUMENTATION** wechseln (oder im Menü **WERKZEUGE** den Befehl **ANSICHT IMPLEMENTIERUNG/DOKUMENTATION** wählen).
- Sie können mit der Funktion **DOKUMENTATION ERZEUGEN** aus dem Menü **WERKZEUGE** die Dokumentation für alle Klassen im Projekt erzeugen.

Sie können weitere Details dazu im Tutorial für BlueJ nachlesen. Sie finden das Tutorial im Menü **HILFE**.

6.11.2 Die Elemente einer Klassendokumentation

Die Dokumentation einer Klasse sollte mindestens die folgenden Punkte enthalten:

- den Klassennamen
- einen Kommentar, der den allgemeinen Zweck und die Eigenschaften der Klasse beschreibt
- eine Versionsnummer
- den (oder die) Autorennamen
- eine Dokumentation für jeden Konstruktor und jede Methode

Die Dokumentation jedes Konstruktors und jeder Methode sollte enthalten:

- den Namen der Methode
- den Ergebnistyp
- die Namen und Typen der Parameter
- eine Beschreibung des Zwecks und der Arbeitsweise der Methode
- eine Beschreibung jedes Parameters
- eine Beschreibung des Ergebnisses

Zusätzlich sollte jedes Projekt einen Projektkommentar enthalten, der oft in einer „ReadMe“-Datei steht. In BlueJ ist dieser Projektkommentar über das Textblatt-Symbol in der linken oberen Ecke des Klassendiagramms zugreifbar.

Übung 6.51 Benutzen Sie die Funktion **DOKUMENTATION ERZEUGEN** in BlueJ, um die Dokumentation für Ihr Kundendienstsystem zu erstellen. Untersuchen Sie das Ergebnis. Ist die Dokumentation korrekt? Ist sie vollständig? Welche Teile sind nützlich, welche eher nicht? Finden Sie Fehler in der Dokumentation?

Einige Elemente der Dokumentation, wie die Namen und die Parameter der Methoden, können immer aus dem Quelltext extrahiert werden. Andere Teile, wie beispielsweise die Kommentare für die Klasse, die Methoden und die Parameter, brauchen mehr Aufmerksamkeit, da sie leicht vergessen werden oder unvollständig oder inkorrekt sind.

In Java werden die Kommentare für **javadoc** mit einer speziellen Zeichenfolge eingeleitet:

```
/**
    Dies ist ein javadoc-Kommentar.
*/
```

Die öffnenden Zeichen für einen Kommentar müssen zwei Sterne enthalten, damit sie von **javadoc** erkannt werden können. Wenn ein solcher Kommentar unmittelbar vor einer Klassendefinition steht, wird er als Klassenkommentar aufgefasst. Wenn er unmittelbar vor der Signatur einer Methode steht, wird er als ein Methodenkommentar aufgefasst.

Die genauen Details, wie die Dokumentation erzeugt und formatiert wird, sind für jede Programmiersprache und jede Programmierumgebung unterschiedlich. Der Inhalt sollte jedoch mehr oder weniger immer der gleiche sein.

Für Java in Verbindung mit **javadoc** stehen einige Schlüsselwörter zur Verfügung, mit denen die Dokumentation formatiert werden kann. Diese Schlüsselwörter beginnen immer mit dem Symbol @:

```
@version
@author
@param
@return
```

Übung 6.52 Finden Sie Beispiele für die Schlüsselwörter von **javadoc** im Quelltext des Projekts *Technischer-Kundendienst*. Wie beeinflussen diese die Formatierung der Dokumentation?

Übung 6.53 Finden und beschreiben Sie weitere Schlüsselwörter von **javadoc**. Ein guter Platz zum Nachforschen ist die Online-Dokumentation der Java-Distribution von Sun Microsystems. Sie bietet ein Dokument mit dem Titel *javadoc – The Java API Documentation Generator* (beispielsweise unter <http://java.sun.com/javase/8/docs/technotes/tools/windows/javadoc.html>). In diesem Dokument werden die Schlüsselwörter *javadoc tags* genannt.

Übung 6.54 Dokumentieren Sie alle Klassen in Ihrer Version des Kundensystems in angemessener Weise.

Konzept

Zugriffsmodifikatoren definieren die Sichtbarkeit eines Datenfelds, eines Konstruktors oder einer Methode. Öffentliche (**public**) Elemente sind sowohl innerhalb der definierenden Klasse zugreifbar als auch von anderen Klassen aus; private (**private**) Elemente sind ausschließlich innerhalb der definierenden Klasse zugreifbar.

6.12 Öffentliche und private Eigenschaften

An dieser Stelle soll ein Aspekt von Klassen etwas näher beleuchtet werden, dem wir bisher schon mehrfach begegnet sind, ohne viel darüber zu sagen: die *Zugriffsmodifikatoren*.

Die beiden Schlüsselwörter **public** oder **private** vor der Deklaration eines Datenfelds und vor der Signatur einer Methode sind Zugriffsmodifikatoren. Beispielsweise:

```
// Datenfeld
private int anzahlSitzplaetze;

// Methoden
public void setzeAlter(int neuesAlter)
{ ...
}

private int berechneDurchschnitt()
{ ...
}
```

Datenfelder, Konstruktoren und Methoden können entweder als öffentlich (**public**) oder als privat (**private**) vereinbart werden, auch wenn wir bisher überwiegend private Datenfelder und öffentliche Methoden und Konstruktoren gesehen haben. Wir werden auf diesen Umstand zurückkommen.

Zugriffsmodifikatoren definieren die Sichtbarkeit eines Datenfelds, Konstruktors oder einer Methode. Wenn eine Methode beispielsweise als **public** vereinbart ist, kann sie innerhalb der definierenden Klasse und von jeder anderen Klasse aufgerufen werden. Private Methoden andererseits können nur innerhalb der Klasse aufgerufen werden, in der sie definiert wurden. Sie sind für andere Klassen nicht sichtbar.

Nachdem wir den Unterschied zwischen der Schnittstelle und der Implementierung einer Klasse diskutiert haben (in Abschnitt 6.3.1), können wir nun sehr viel leichter den Sinn dieser Schlüsselwörter verstehen.

Sie erinnern sich: Die Schnittstelle einer Klasse ist die Menge der Details, die ein anderer Programmierer, der die Klasse benutzen will, benötigt. Sie liefert Informationen darüber, wie die Klasse benutzt werden kann. Die Schnittstelle schließt die Konstruktoren, die Signaturen der Methoden und deren Kommentare mit ein. Sie wird auch als der *öffentliche* Teil einer Klasse bezeichnet. Ihr Zweck ist zu definieren, was die Klasse tut.

Die Implementierung ist der Teil der Klasse, der genau festlegt, *wie* die Klasse ihre Aufgabe erfüllt. Die Methodenrümpfe mit den Java-Anweisungen und die meisten Datenfelder sind Teil der Implementierung. Die Implementierung wird auch als der *private* Teil einer Klasse bezeichnet. Der Benutzer einer Klasse muss die Implementierung nicht kennen. Tatsächlich gibt es gute Gründe, weshalb ein Benutzer davor geschützt werden sollte, etwas über die Implementierung zu wissen (oder zumindest von diesem Wissen Gebrauch zu machen). Dieses Prinzip wird auch als das *Geheimnisprinzip* bezeichnet (*information hiding*).

Das Schlüsselwort **public** deklariert für ein Element einer Klassendefinition (ein Datenfeld oder eine Methode), dass es Teil der Schnittstelle ist (also öffentlich sichtbar); das Schlüsselwort **private** deklariert es als einen Teil der Implementierung (also als verborgen vor dem Zugriff von außen).

6.12.1 Das Geheimnisprinzip

In vielen objektorientierten Sprachen werden die Interna einer Klasse – ihre Implementierung – vor anderen Klassen verborgen. Dies hat zwei Aspekte: Erstens sollte ein die Klasse benutzender Programmierer keine Informationen über ihren internen Aufbau benötigen; zweitens sollte ihm nicht erlaubt sein, diese internen Details zu kennen.

Das erste Prinzip – keine Kenntnis über Interna benötigen – hat mit Abstraktion und Modularisierung zu tun, wie in Kapitel 3 diskutiert. Wenn es notwendig wäre, alle Interna von allen benötigten Klassen zu kennen, dann würden wir große Systeme niemals fertig bekommen.

Das zweite Prinzip – keine Kenntnis über Interna erlauben – hat eine andere Bedeutung. Es hat zwar auch mit Modularisierung zu tun, aber in einem anderen Zusammenhang. Die Programmiersprache erlaubt nicht den Zugriff auf die privaten Anteile einer Klasse in den Anweisungen einer anderen Klasse. Dies garantiert, dass eine Klasse nicht davon abhängt, dass eine andere Klasse exakt auf diese Weise implementiert ist.

Dieser Aspekt ist sehr wichtig für die Wartung. Es kommt häufig vor, dass ein Wartungsprogrammierer die Implementierung einer Klasse ändern oder erweitern muss, um sie zu verbessern oder um Fehler zu beheben. Idealerweise sollte eine Änderung in der Implementierung einer Klasse nicht auch eine Änderung in einer anderen Klasse notwendig machen. Dieser Aspekt wird auch *Kopplung* genannt: Wenn eine Änderung an einer Stelle eines Programms nicht auch Änderungen an anderen Stellen impliziert, dann wird dies schwache oder lose Kopplung genannt. Lose Kopplung ist gut, denn sie erleichtert die Aufgaben eines Wartungsprogrammierers sehr. Anstatt viele Klasse verstehen und ändern zu müssen, muss er nur eine Klasse verstehen und ändern. Wenn beispielsweise ein Java-Programmierer eine Verbesserung an der Implementierung der Klasse **ArrayList** vornimmt, dann hoffen wir sehr, dass deshalb nicht alle Klassen geändert werden müssen, die diese Klasse benutzen. Das funktioniert deshalb, weil wir in unseren Quelltexten keinen Bezug auf die Implementierung der Klasse **ArrayList** nehmen.

Um es etwas genauer zu sagen: Die Regel – keine Kenntnis über Interna erlauben – bezieht sich nicht auf den Programmierer einer benutzenden Klasse, sondern auf die benutzende Klasse selbst. Es ist üblicherweise kein Problem, wenn ein Programmierer die Details einer anderen Klasse kennt, aber die von ihm implementierte Klasse sollte keine Kenntnis über die Interna haben, also nicht von den Interna einer anderen Klasse abhängen. Der Programmierer der beiden beteiligten Klassen kann sogar derselbe sein, aber die Klassen sollten lose gekoppelt sein.

Die Konzepte lose Kopplung und Geheimnisprinzip sind sehr wichtig, und wir werden in späteren Kapiteln auf sie zurückkommen.

Konzept

Das **Geheimnisprinzip** besagt, dass die internen Details der Implementierung einer Klasse vor anderen Klassen verborgen sein sollten. Dies unterstützt eine bessere Modularisierung von Anwendungen.

An dieser Stelle ist wichtig zu verstehen, dass das Schlüsselwort **private** das Geheimnisprinzip dadurch erzwingt, dass andere Klassen keinen Zugriff auf diese privaten Anteile haben können. Dies ermöglicht lose Kopplung und macht eine Anwendung modularer und leichter zu warten.

6.12.2 Private Methoden und öffentliche Datenfelder

Die meisten Methoden, die wir bisher betrachtet haben, waren öffentlich. Dies ermöglicht es anderen Klassen, diese Methoden aufzurufen. An manchen Stellen haben wir allerdings schon private Methoden benutzt. In der Klasse **Kundendienstsystem** im Projekt *Technischer-Kundendienst* beispielsweise haben wir die Methoden **willkommenstextAusgeben** und **abschiedstextAusgeben** als privat deklariert.

Der Grund, beide Optionen für Methoden zu verwenden, liegt darin, dass Methoden tatsächlich für sehr unterschiedliche Zwecke eingesetzt werden. Sie werden eingesetzt, um den Benutzern einer Klasse Operationen anzubieten (öffentliche Methoden), und sie werden benutzt, um eine größere Aufgabe in kleinere zu zerlegen, um damit die größere Aufgabe leichter handhabbar zu machen. Im zweiten Fall sind die Unteraufgaben nicht dafür vorgesehen, von außerhalb der Klasse aufgerufen zu werden, sondern sie werden nur für eine bessere interne Strukturierung der Klasse in eigenen Methoden definiert. In einem solchen Fall sollten diese Methoden privat deklariert werden. **willkommenstextAusgeben** und **abschiedstextAusgeben** sind Beispiele für diesen Fall.

Ein weiterer guter Grund für private Methoden ist, wenn eine bestimmte Dienstleistung (als eine Unteraufgabe) von mehreren Methoden in einer Klasse benötigt wird. Statt diese Anweisungen mehrfach zu schreiben, können wir eine private Methode mit diesen Anweisungen schreiben und diese Methode dann von unterschiedlichen Stellen aus aufrufen. Wir werden später noch ein Beispiel für diesen Fall sehen.

In Java können auch Datenfelder als privat oder als öffentlich deklariert werden. Bisher haben wir kein Beispiel für öffentliche Datenfelder gesehen, und das hat seinen Grund: Öffentliche Datenfelder brechen nämlich mit dem Geheimnisprinzip. Durch sie wird eine Klasse, die sich auf diese Informationen verlässt, verletzlich für Änderungen der Implementierung. Auch wenn Java erlaubt, dass Datenfelder als öffentlich deklariert werden, betrachten wir dies als schlechten Stil und machen von der Möglichkeit keinen Gebrauch. Einige andere objektorientierte Sprachen erlauben öffentliche Datenfelder gar nicht.

Ein weiterer Grund für private Datenfelder ist, dass ein Objekt damit größere Kontrolle über seinen Zustand hat. Wenn die Zugriffe auf private Datenfelder ausschließlich über die sondierenden und verändernden Methoden möglich sind, dann kann ein Objekt garantieren, dass kein Datenfeld auf einen Wert gesetzt wird, der mit dem Gesamtzustand des Objekts nicht vereinbar ist. Dieser Grad an Integrität kann mit öffentlichen Datenfeldern nicht garantiert werden.

Kurz gesagt: Datenfelder sollten immer privat sein.

Java definiert noch zwei weitere Zugriffsstufen. Die eine wird mit dem Schlüsselwort **protected** deklariert, die andere wird wirksam, wenn gar kein Modifikator angegeben ist. Wir werden diese in späteren Kapiteln besprechen.

6.13 Klassen über ihre Schnittstelle verstehen

Wir werden hier kurz ein weiteres Projekt besprechen, um die in diesem Kapitel vorgestellten Konzepte zu vertiefen und Erfahrungen zu sammeln. Das Projekt trägt den Namen *Kritzeln* und ist im Buchprojekte-Ordner zu Kapitel 6 zu finden. Dieser Abschnitt führt keine neuen Konzepte ein; es besteht zum großen Teil aus Übungen, in die vereinzelte Kommentare eingestreut sind.

6.13.1 Das Demo-Projekt Kritzeln

Das Projekt *Kritzeln* enthält drei Klassen: **ZeichnenDemo**, **Stift** und **Canvas** (Abbildung 6.4).

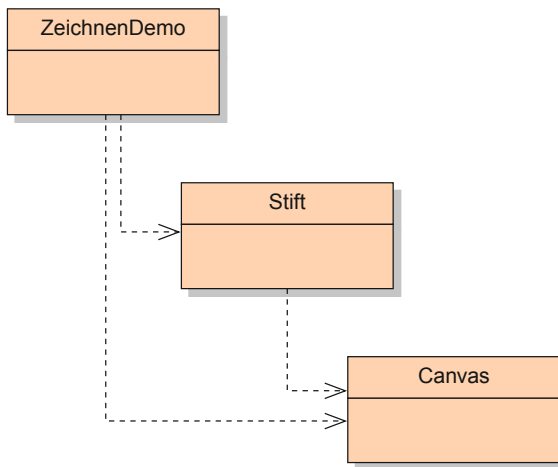


Abbildung 6.4
Das Projekt *Kritzeln*.

Die Klasse **Canvas** (englisch für „Leinwand“) bietet ein Fenster auf dem Bildschirm an, auf dem gezeichnet werden kann. Sie bietet Operationen zum Zeichnen von Linien, Figuren und Text. Ein **Canvas** kann benutzt werden, indem interaktiv oder von einem anderen Objekt aus eine Instanz erzeugt wird. Diese Klasse sollte keiner Veränderungen bedürfen. Es ist wahrscheinlich am besten, sie als Bibliotheksklasse zu behandeln.⁴ Öffnen Sie den Editor der Klasse und wechseln Sie in die Ansicht *Dokumentation*. Dies zeigt die Schnittstelle der Klasse im **javadoc**-Format.

⁴ Anmerkung des Übersetzers: Aufgrund dieser Rolle als Bibliotheksklasse wurde diese Klasse bewusst nicht übersetzt.

Die Klasse **Stift** liefert das **Stift**-Objekt, das dazu verwendet werden kann, durch Bewegen des Stiftes auf dem Bildschirm zu zeichnen. Der Stift selbst ist unsichtbar, zieht aber eine Linie, wenn er über die Leinwand bewegt wird.

Die Klasse **ZeichnenDemo** bietet mehrere kleine Beispiele für die Verwendung eines Stift-Objekts für grafische Ausgaben.

Der beste Einstiegspunkt in dieses Projekt ist die Klasse **ZeichnenDemo**.

Übung 6.55 Erzeugen Sie ein Objekt der Klasse **ZeichnenDemo** und experimentieren Sie mit seinen diversen Methoden. Lesen Sie den Quelltext von **ZeichnenDemo** und beschreiben Sie (schriftlich) die Funktionsweise einer jeden Methode.

Übung 6.56 Erzeugen Sie interaktiv ein **Stift**-Objekt unter Verwendung seines Standardkonstruktors (der Konstruktor ohne Parameter). Experimentieren Sie mit seinen Methoden. Achten Sie darauf, dass Sie währenddessen ein Fenster geöffnet haben, das Ihnen die Dokumentation der Klasse **Stift** zeigt (entweder das Editorfenster in der Dokumentationsansicht oder ein Webbrowserfenster, das die Projektdokumentation zeigt). Vergewissern Sie sich in der Dokumentation, dass Sie verstehen, was jede Methode macht.

Übung 6.57 Erzeugen Sie interaktiv eine Instanz der Klasse **Canvas** und versuchen Sie sich an einigen ihrer Methoden. Auch hier sollten Sie dabei Bezug nehmen auf die Dokumentation der Klasse.

Einige der Methoden in den Klassen **Stift** und **Canvas** beziehen sich auf Parameter vom Typ **Color**. Dieser Typ ist in der Klasse **Color** im Paket **java.awt** definiert (das heißt, sein voll qualifizierter Name lautet **java.awt.Color**). Die Klasse **Color** definiert einige Farbkonstanten, die wir wie folgt ansprechen können:

`Color.RED`

Um diese Konstanten zu verwenden, müssen Sie die Klasse **Color** in der verwendeten Klasse importieren.

Übung 6.58 Finden Sie einige Einsatzbereiche von Farbkonstanten im Quelltext der Klasse **ZeichnenDemo**.

Übung 6.59 Schreiben Sie vier weitere Farbkonstanten nieder, die in der Klasse **Color** verfügbar sind. Lesen Sie in der Klassendokumentation nach, um herauszufinden, welche Konstanten es gibt.

Wenn wir interaktiv Methoden aufrufen, die Parameter der Klasse **Color** erwarten, müssen wir ein wenig anders auf die Klasse Bezug nehmen. Da der interaktive Dia-

log keine **import**-Anweisung erlaubt (und damit die Klasse **Color** nicht automatisch bekannt ist), müssen wir den voll qualifizierten Klassennamen schreiben, um auf die Klasse Bezug zu nehmen (Abbildung 6.5). Auf diese Weise kann das Java-System die Klasse finden, ohne dass eine **import**-Anweisung verwendet werden muss.

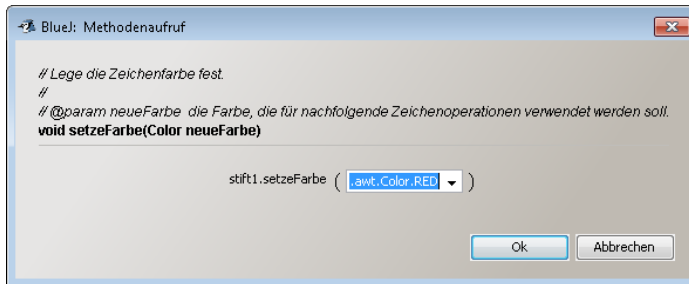


Abbildung 6.5
Ein Methodenaufruf
mit einem voll qualifi-
zierten Klassennamen.

Da wir inzwischen wissen, wie wir die Farbe für Stifte und Leinwände ändern, können wir uns weiteren Übungen zuwenden.

Übung 6.60 Erzeugen Sie ein **Canvas**. Zeichnen Sie unter interaktiver Verwendung der Methoden von **Canvas** einen roten Kreis ungefähr in der Mitte der Leinwand und anschließend ein gelbes Rechteck.

Übung 6.61 Wie löschen Sie die ganze Leinwand?

Wie Sie gesehen haben, können Sie entweder direkt auf die Leinwand zeichnen oder sich dazu eines **Stift**-Objekts bedienen. Der Stift bietet uns eine Abstraktion, die eine aktuelle Position, Rotation und Farbe hält, was die Erzeugung einiger Arten von Zeichnungen einfacher macht. Lassen Sie uns noch ein wenig herumexperimentieren. Dieses Mal wollen wir jedoch Code in einer Klasse schreiben, anstatt interaktive Aufrufe zu verwenden.

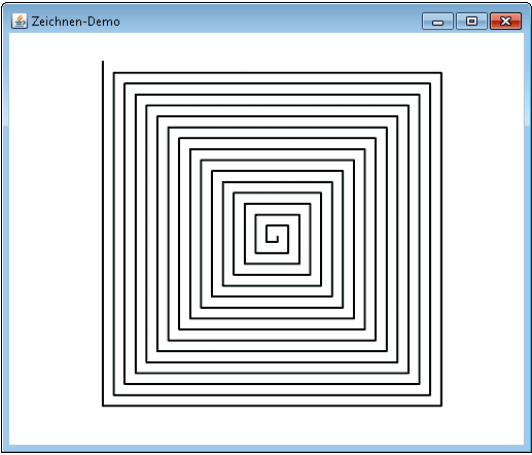
Übung 6.62 Erzeugen Sie in der Klasse **ZeichnenDemo** eine neue Methode namens **dreieckZeichnen**. Diese Methode sollte einen Stift erzeugen (wie in der Methode **quadratZeichnen**) und dann ein grünes Dreieck zeichnen.

Übung 6.63 Schreiben Sie eine Methode **fuenfeckZeichnen**, die ein Fünfeck zeichnet.

Übung 6.64 Schreiben Sie eine Methode **polygonZeichnen(int n)**, die ein gleichmäßiges Polygon mit **n** Seiten zeichnet (d.h., **n=3** zeichnet ein Dreieck, **n=4** zeichnet ein Quadrat usw.).

Übung 6.65 Schreiben Sie eine Methode namens **spirale**, die eine Spirale zeichnet (siehe Abbildung 6.6).

Abbildung 6.6
Eine Spirale, die
auf eine Canvas
gezeichnet ist.

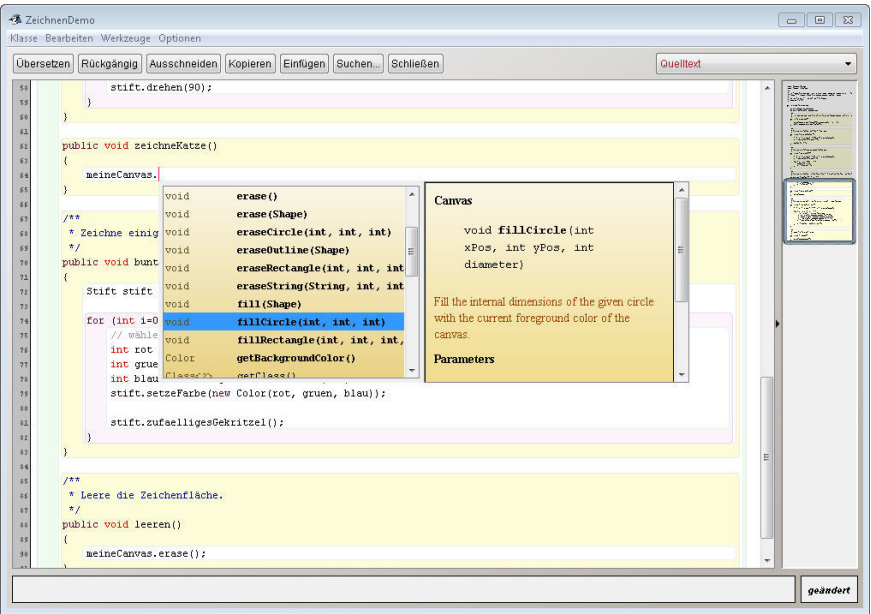


6.13.2 Code-Vervollständigung

Oft sind wir mit der Bibliotheksklasse, die wir benutzen, einigermaßen vertraut. Dennoch können wir uns nicht immer genau an alle Namen ihrer Methoden oder der Parameter erinnern. Für solche Fälle bieten Entwicklungsumgebungen in der Regel Unterstützung in Form von Code-Vervollständigung.

Code-Vervollständigung ist eine Funktion, die im BlueJ-Editor zur Verfügung steht, wenn der Cursor hinter dem Punkt eines Methodenaufrufs steht. Wenn Sie dann `[Strg]+[Leertaste]` drücken, erscheint ein Kontextmenü, das alle Methoden in der Schnittstelle des Objekts auflistet, das wir im Aufruf verwenden (Abbildung 6.7).

Abbildung 6.7
Code-Vervoll-
ständigung.



Wenn das Kontextmenü zur Code-Vervollständigung eingeblendet wird, können wir den Anfang des Methodennamens eintippen und damit die Anzahl der ange-

zeigten Methoden in der Liste reduzieren. Durch Betätigen der Eingabetaste übernehmen Sie den ausgewählten Methodenaufruf in Ihren Quelltext. Die Code-Vervollständigung kann auch ohne ein vorangehendes Objekt verwendet werden, um lokale Methoden aufzurufen.

Die Code-Vervollständigung sollte kein Ersatz für das Lesen der Klassendokumentation sein, da sie nicht alle Informationen umfasst (wie der einführende Klassenkommentar). Doch sobald wir eine Klasse recht gut kennen, ist die Code-Vervollständigung eine große Hilfe, um sich die Einzelheiten einer Methode schnell in Erinnerung zu rufen und den Aufruf in den Quelltext zu übernehmen.

Übung 6.66 Geben Sie `myCanvas.er` in Ihrer Klasse `ZeichnenDemo` ein und drücken Sie dann `[Strg]` + `[Leertaste]` (wobei der Cursor unmittelbar hinter dem eingegebenen Text steht), um die Code-Vervollständigung zu aktivieren. Wie viele Methoden werden angezeigt?

Übung 6.67 Fügen Sie Ihrer Klasse `ZeichnenDemo` eine Methode hinzu, die direkt ein Bild auf der Leinwand erzeugt (ohne ein `Stift`-Objekt zu verwenden). Das Bild kann irgendetwas Beliebigen sein, sollte aber mindestens einige Formen, verschiedene Farben und Text enthalten. Nutzen Sie Code-Vervollständigung, während Sie Ihren Quelltext eingeben.

6.13.3 Das Demo-Projekt Baelle

Öffnen Sie das Projekt *Baelle* und stellen Sie fest, was es macht. Erzeugen Sie ein `BallDemo`-Objekt und führen Sie seine Methode `springenLassen` aus.

Übung 6.68 Ändern Sie die Methode `springenLassen` in der Klasse `BallDemo`, damit der Benutzer wählen kann, wie viele Bälle springen sollen.

Für diese Übungen sollten Sie zur Speicherung der Bälle eine Sammlung verwenden. So kann die Methode mit jeder beliebigen Anzahl Bälle – seien es 1, 3 oder 75 – ausgeführt werden. Die Bälle sollten am Anfang nebeneinander oben auf der Leinwand angezeigt werden.

Welche Art Sammlung sollten wir wählen? Bis jetzt haben wir `ArrayList`, `HashMap` und `HashSet` kennengelernt. Machen Sie erst die nächsten zwei Übungen, bevor Sie Ihre Implementierung schreiben.

Übung 6.69 Welche Art Sammlung (`ArrayList`, `HashMap` und `HashSet`) ist am besten geeignet, um die Bälle für die neue Methode `springenLassen` zu speichern? Halten Sie Ihre Antwort schriftlich fest und begründen Sie Ihre Wahl.

Übung 6.70 Ändern Sie die Methode `springenLassen`, um die Bälle in der oberen Hälfte des Bildschirms nach dem Zufallsprinzip anzuordnen.

Übung 6.71 Schreiben Sie eine neue Methode namens `kastenSpringen`. Diese Methode zeichnet ein Rechteck (den „Kasten“) auf den Bildschirm sowie einen oder mehrere Bälle in diesen Kasten. Verwenden Sie für die Bälle nicht die Klasse `Ball`, sondern erzeugen Sie eine neue Klasse `KastenBall`, wobei sich der Ball im Kasten bewegt und von den Wänden abprallt, sodass der Ball immer im Kasten bleibt. Die Anfangsposition sowie die Geschwindigkeit des Balls sollten Zufallswerte sein. Die Methode `kastenSpringen` sollte einen Parameter übernehmen, der angibt, wie viele Bälle im Kasten erzeugt werden sollen.

Übung 6.72 Zeichnen Sie die Bälle in `kastenSpringen` mit Zufallsfarben.

6.14 Klassenvariablen und Konstanten

Bisher haben wir die Klasse `Ball` nicht näher betrachtet. Wenn Sie verstehen möchten, wie die Animation funktioniert, sollten Sie sich auch diese Klasse genauer ansehen. Sie ist recht einfach – die einzige etwas kompliziertere Methode ist `bewegen`, wobei der Ball seine neue Position auf seiner Bahn bekommt.

Wir überlassen es weitgehend Ihnen selbst, diese Methode zu verstehen, bis auf ein Detail, das wir hier ansprechen möchten. Wir beginnen mit einer Übung.

Übung 6.73 In der Klasse `Ball` finden Sie die Definition der Gravitation (eine einfache ganze Zahl). Vergrößern oder verkleinern Sie diesen Wert, übersetzen Sie und starten Sie die Demo erneut. Beobachten Sie eine Veränderung?

Das interessanteste Detail in dieser Klasse ist die Zeile

```
private static final int GRAVITATION = 3;
```

Dies ist ein Konstrukt, das wir bisher noch nicht gesehen haben. Tatsächlich werden hier zwei neue Schlüsselwörter eingesetzt, die gemeinsam verwendet werden: `static` und `final`.

Konzept

Auch Klassen können Datenfelder haben. Diese werden **Klassenvariablen** oder auch statische Variablen genannt. Von einer Klassenvariablen existiert immer nur genau eine Kopie, unabhängig von der Anzahl der erzeugten Instanzen.

6.14.1 Das Schlüsselwort `static`

Mit dem Schlüsselwort `static` werden in Java *Klassenvariablen* definiert. Klassenvariablen sind Datenfelder, die in der Klasse selbst, nicht in den Objekten gespeichert werden. Dies unterscheidet sie fundamental von den Instanzvariablen (die Datenfelder, die wir bisher benutzt haben). Betrachten Sie folgenden Quelltextabschnitt (ein Teil der Klasse `Ball`):

```
public class Ball
{
    // Einfluss der Gravitation
    private static final int GRAVITATION = 3;

    private int xPosition;
    private int yPosition;
```

```

    andere Datenfelder und Methoden hier ausgelassen
}

```

Nehmen Sie nun an, dass wir drei Instanzen der Klasse **Ball** erzeugen. Die resultierende Situation ist in Abbildung 6.8 dargestellt.

Aus dem Diagramm können wir ersehen, dass die Instanzvariablen (**xPosition** und **yPosition**) in jedem Objekt gespeichert sind. Da wir drei Objekte erzeugt haben, haben wir auch drei voneinander unabhängige Kopien dieser Variablen.

Die Klassenvariable **GRAVITATION** hingegen ist in der Klasse selbst gespeichert. Als Konsequenz gibt es immer nur genau eine Kopie dieser Variablen, unabhängig von der Anzahl der erzeugten Instanzen.

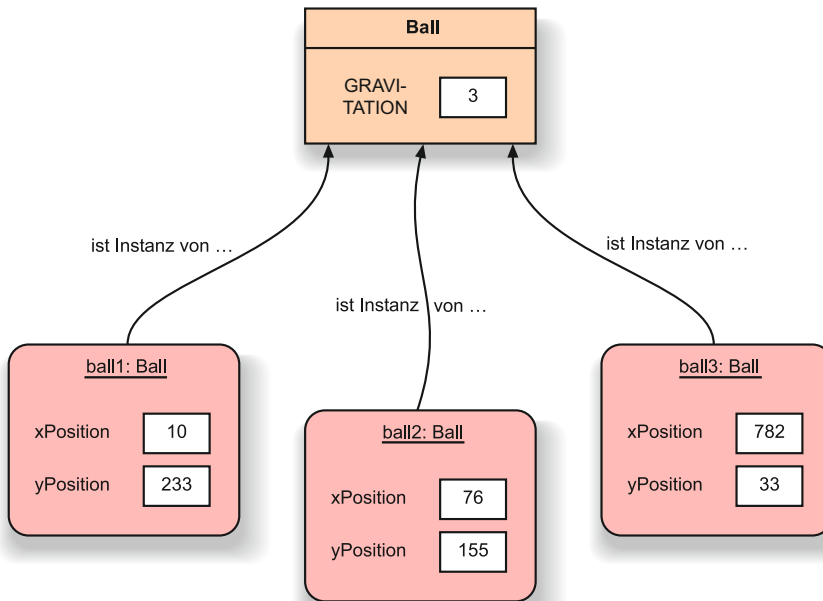


Abbildung 6.8
Instanzvariablen und
eine Klassenvariable.

Im Quelltext der Klasse kann auf diese Art der Variablen (lesend und schreibend) genauso zugegriffen werden wie auf eine Instanzvariable. Auf die Klassenvariable kann auch in allen Instanzen der Klasse zugegriffen werden. Die Konsequenz ist, dass die Objekte diese Variable gemeinsam benutzen.

Klassenvariablen werden oft dann eingesetzt, wenn ein Wert für alle Instanzen einer Klasse gleich sein soll. Anstatt diesen gleichen Wert in jedem einzelnen Objekt abzuliegen (eine Verschwendung von Platz und möglicherweise schwierig zu koordinieren), kann ein einzelner Wert von allen Instanzen geteilt werden.

Java unterstützt auch Klassenmethoden (auch als *statische Methoden* bekannt), also Methoden, die zu einer Klasse gehören. Wir werden diese später in einem eigenen Abschnitt diskutieren.

6.14.2 Konstanten

Das Schlüsselwort **static** wird häufig bei der Definition von *Konstanten* verwendet. Konstanten sind ähnlich wie Variablen, können aber ihren Wert während der Ausführung einer Anwendung nicht mehr ändern. In Java werden Konstanten mit dem Schlüsselwort **final** definiert. Beispielsweise:

```
private final int GROESSE = 10;
```

Hier definieren wir eine Konstante **GROESSE** mit dem Wert 10. Wir stellen fest, dass Konstantendeklarationen Variablendeklarationen sehr ähnlich sind, mit zwei Unterschieden:

- Sie enthalten das Schlüsselwort **final** vor dem Typnamen.
- Sie müssen bei ihrer Deklaration auch unmittelbar initialisiert werden.

Wenn eine Variable ihren Wert nicht ändern soll, dann sollte sie mit **final** deklariert werden. Dies garantiert, dass sie später nicht aus Versehen geändert werden kann. Der Versuch, ein konstantes Datenfeld zu ändern, führt zu einer Fehlermeldung bei der Übersetzung. Konstanten werden nach Konvention häufig in Großbuchstaben geschrieben. Wir folgen dieser Konvention in diesem Buch.

In der Praxis kommt es häufig vor, dass Konstanten für alle Instanzen einer Klasse gelten sollen. In solchen Situationen verwendet man *Klassenkonstanten*. Klassenkonstanten sind konstante Datenfelder einer Klasse. Ein Beispiel:

```
private static final int GROESSE = 10;
```

Die Definition von **GRAVITATION** in unserem Projekt *Baelle* ist ein Beispiel für eine solche Konstante. Dies ist die Art, in der Konstanten meist benutzt werden. Instanzbezogene Konstanten kommen sehr viel seltener vor.

Wir haben zwei weitere Beispiele für Konstanten in dem Projekt *Kritzeln* kennengelernt. Das erste Beispiel besteht aus zwei Konstanten in der Klasse **Stift**, die die Größe des Zufallsgekritzel definieren (kehren Sie zurück zum Projekt und suchen Sie danach!). Das zweite Beispiel war die Verwendung der Farbkonstanten in diesem Projekt wie **Color.RED**. In diesem Fall haben wir die Konstanten nicht definiert, sondern Konstanten verwendet, die in einer anderen Klasse definiert sind.

Der Grund, warum wir die Konstanten der Klasse **Color** verwenden konnten, ist, dass wir sie als **public** deklariert haben. Im Gegensatz zu anderen Datenfeldern (die, wie wir vorher berichtet haben, niemals als **public** deklariert werden sollten) ist das Deklarieren von öffentlichen Konstanten im Allgemeinen unproblematisch und manchmal nützlich.

Übung 6.74 Schreiben Sie Konstantendeklarationen für folgende Datenfelder:

- ein öffentliches Datenfeld für eine Toleranz bei Messwerten mit dem Wert 0.001;
- ein privates Datenfeld für den Prozentsatz zum Bestehen einer Klausur mit dem Wert 40;
- ein öffentliches Datenfeld vom Typ **char**, das den Buchstaben „h“ für das Hilfe-Kommando festlegt.

Übung 6.75 Welche Konstantennamen werden in der Klasse `java.lang.Math` definiert?

Übung 6.76 Geben Sie Gründe dafür an, warum es sinnvoll ist, in einem Programm, das die Konstante 73,28166 an zehn verschiedenen Orten verwendet, diesen Wert mit einem Variablennamen zu verknüpfen.

6.15 Klassenmethoden

Alle bisher betrachteten Methoden waren *Instanzmethoden*: Sie werden an einer Instanz einer Klasse aufgerufen. Wir unterscheiden Klassenmethoden von Instanzmethoden dadurch, dass Klassenmethoden ohne eine Instanz aufgerufen werden können – die Klasse selbst reicht aus.

6.15.1 Statische Methoden

In Abschnitt 6.14 haben wir Klassenvariablen diskutiert. Klassenmethoden sind konzeptuell verwandt und benutzen eine verwandte Syntax (das Schlüsselwort **static** in Java). Genau wie Klassenvariablen zu einer Klasse statt zu einer Instanz gehören, gehören auch Klassenmethoden zu einer Klasse.

Eine Klassenmethode wird definiert, indem das Schlüsselwort **static** vor den Typnamen in einer Methodensignatur gestellt wird:

```
public static int gibAnzahlErzeugterInstanzen()
{
    ...
}
```

Solch eine Methode kann dann aufgerufen werden, indem in der üblichen Punkt-schreibweise dem Punkt der Name der definierenden Klasse vorangestellt wird. Wenn diese Methode beispielsweise in der Klasse **Gegenstand** definiert ist, dann kann sie mit folgender Anweisung aufgerufen werden:

```
int anzahlGegenstände = Gegenstand.gibAnzahlErzeugterInstanzen();
```

Beachten Sie, dass der Name der Klasse vor dem Punkt steht – es musste hier kein Objekt erzeugt werden, um die Methode aufzurufen.

Übung 6.77 Lesen Sie die Dokumentation der Klasse **Math** im Paket `java.lang`. Sie enthält viele statische Methoden. Finden Sie die Methode, die das Maximum zweier ganzer Zahlen ermittelt. Wie sieht ihr Kopf aus?

Übung 6.78 Was denken Sie, warum diese Methoden in der Klasse **Math** statisch sind? Könnten sie auch als Instanzmethoden geschrieben werden?

Übung 6.79 Schreiben Sie eine Testklasse mit einer Methode, die das Zählen in einer Schleife von 1 bis 10000 misst. Sie können dazu die Methode `currentTimeMillis` aus der Klasse `System` verwenden, die bei Zeitmessungen helfen kann.

Übung 6.80 Kann eine statische Methode aus einer Instanzmethode heraus aufgerufen werden? Kann eine Instanzmethode aus einer statischen Methode heraus aufgerufen werden? Kann eine statische Methode von einer anderen statischen Methode aufgerufen werden? Beantworten Sie diese Fragen schriftlich, erzeugen Sie dann ein Testprojekt, um Ihre Antworten zu überprüfen, und probieren Sie sie aus. Erklären Sie Ihre Antworten und Beobachtungen detailliert.

Übung 6.81 Die Klasse `Collections` in den `java.util`-Paketen enthält eine Vielzahl an statischen Methoden, die mit Sammlungen wie `ArrayList`, `LinkedList`, `HashMap` und `HashSet` benutzt werden können. Lesen Sie beispielsweise die Klassendokumentation für die Methoden `min`, `max` und `sort`. Auch wenn Sie eventuell die Erklärungen oder die Notation nicht vollständig verstehen, wird das Wissen darum, dass es diese Methoden gibt, für Sie schon hilfreich sein.

6.15.2 Einschränkungen für Klassenmethoden

Weil Klassenmethoden mit einer Klasse statt mit einer Instanz verknüpft sind, gelten zwei wichtige Einschränkungen für sie. Die erste Einschränkung ist, dass eine Klassenmethode nicht auf Instanzvariablen zugreifen kann, die innerhalb der Klasse definiert sind. Das ist logisch, denn Instanzvariablen sind mit einzelnen Instanzen verknüpft. Klassenmethoden können lediglich auf die Klassenvariablen ihrer Klasse zugreifen. Die zweite Einschränkung ist der ersten ähnlich: Eine Klassenmethode kann keine Instanzmethoden ihrer Klasse aufrufen. Eine Klassenmethode kann ausschließlich andere Klassenmethoden ihrer Klasse aufrufen.

Sie werden feststellen, dass wir in den Beispielen in diesem Buch nur sehr wenig Gebrauch von Klassenmethoden machen.

6.16 Programmausführung ohne BlueJ

Wenn wir ein Programm fertig implementiert haben, möchten wir es möglicherweise an andere zum Ausprobieren weitergeben. Dazu wäre es schön, wenn andere das Programm ausführen können, ohne BlueJ starten zu müssen. Um dies tun zu können, müssen wir ein weiteres Konzept einführen: Klassenmethoden, die in Java auch statische Methoden genannt werden.

6.16.1 Die Methode `main`

Wenn wir eine Java-Anwendung ohne BlueJ starten wollen, dann benötigen wir dazu eine Klassenmethode. In BlueJ erzeugen wir typischerweise ein Objekt und

rufen eine seiner Methoden auf, aber ohne BlueJ muss eine Anwendung auch ohne ein existierendes Objekt gestartet werden können. Klassen sind das Einzige, das von Anfang an „da“ ist, entsprechend muss die erste aufzurufende Methode eine Klassenmethode sein.

Die Java-Definition zum Starten einer Anwendung ist recht simpel: Der Benutzer gibt die Klasse an, die den Startpunkt bilden soll, und das Java-System ruft an dieser Klasse die Methode `main` auf. Diese Methode muss eine fest definierte Signatur haben:

```
public static void main(String[] args)
```

Wenn eine solche Methode in der Klasse nicht existiert, wird eine Fehlermeldung ausgegeben. Anhang E beschreibt die Details dieser Methode und die notwendigen Befehle, um ein Java-Programm ohne BlueJ starten zu können.

Übung 6.82 Fügen Sie eine `main`-Methode Ihrer Klasse `Kundendienstsystem` im Projekt *Technischer-Kundendienst* hinzu. Diese Methode sollte ein Objekt der Klasse `Kundendienstsystem` erzeugen und an diesem die Methode `starten` aufrufen. Testen Sie die Methode `main`, indem Sie sie innerhalb von BlueJ aufrufen. Klassenmethoden können aus dem Kontextmenü einer Klasse aufgerufen werden.

Übung 6.83 Starten Sie Ihr Programm ohne BlueJ.

Übung 6.84 Kann eine Klasse zählen, wie viele Instanzen von dieser Klasse erzeugt wurden? Was benötigt man dazu? Schreiben Sie einige Codefragmente, mit denen Sie die notwendigen Schritte illustrieren. Nehmen Sie an, Sie möchten eine statische Methode namens `anzahlInstanzen` haben, die die Anzahl der erzeugten Instanzen zurückgibt.

6.17 Weitere fortgeschrittene Konzepte

Im vorigen Kapitel haben wir Konzepte eingeführt, die wir als „fortgeschritten“ bezeichnet haben, und vorgeschlagen, diese Inhalte beim ersten Lesen eventuell zu überspringen. In diesem Abschnitt bieten wir weitere vertiefende Inhalte an – und geben denselben Rat.

6.17.1 Polymorphe Sammlungstypen (fortgeschrittene Konzepte)

Bisher haben wir mehrere unterschiedliche Sammlungsklassen aus dem `java.util`-Paket vorgestellt, wie `ArrayList`, `HashMap` und `HashSet`. Jede Klasse hat einige charakteristische Funktionen, die sie für bestimmte Situationen geeignet macht. Die Klassen besitzen jedoch auch sehr viele Ähnlichkeiten und gemeinsame Methoden. Dies legt nahe, dass sie auf gewisse Weise miteinander verknüpft sind. In der objektorientierten Programmierung werden Beziehungen zwischen Klassen über das Kon-

zept der *Vererbung* ausgedrückt. Wir werden dieses Thema in den Kapiteln 10 und 11 ausführlich behandeln, aber es ist sinnvoll, es hier kurz anzureißen, um Ihr Verständnis für die unterschiedlichen Arten von Sammlungen zu vertiefen.

Wenn wir verstehen wollen, wie die unterschiedlichen Sammlungsklassen zu verwenden sind, ist es hilfreich, deren Namen heranzuziehen. Es scheint nahezu liegen, aus dem Namen abzuleiten, dass ein **HashSet** in ähnlicher Weise zu benutzen ist wie eine **HashMap**. Tatsächlich haben wir jedoch gesehen, dass ein **HashSet** in seiner Benutzung sehr viel enger mit einer **ArrayList** verwandt ist. Die Namen bestehen aus zwei Teilen, zum Beispiel „Array“ und „List“. Die zweite Hälfte gibt einen Hinweis auf die *Art der Sammlung*, mit der wir es zu tun haben (**List**, **Map**, **Set**), und die erste Hälfte teilt uns mit, *wie sie implementiert ist*, zum Beispiel mittels eines Arrays (wird in Kapitel 7 besprochen) oder durch Hashing (wird in Kapitel 11 behandelt).

Bei der Programmierung mit Sammlungen ist die Art der Sammlung (der zweite Teil) am wichtigsten. Wir haben bereits angesprochen, dass wir von der Implementierung abstrahieren können; wir müssen nicht groß darüber nachdenken. So gesehen sind sich für unsere Zwecke **HashSet** und **TreeSet** sehr ähnlich. Beides sind Mengen, daher verhalten sie sich entsprechend gleich. Der größte Unterschied liegt in ihrer Implementierung, was nur eine Rolle spielt, wenn Effizienz ins Spiel kommt: Eine Implementierung wird einige Operationen viel schneller durchführen als die andere. Wir werden uns den Fragen der Effizienz allerdings erst später zuwenden und nur, wenn wir es mit sehr großen Sammlungen oder Anwendungen zu tun haben, für die die Leistung kritisch ist.

Eine Konsequenz aus dieser Ähnlichkeit ist, dass wir oft Details der Implementierung ignorieren können, wenn wir eine Variable deklarieren, die sich auf ein bestimmtes Sammlungsobjekt bezieht. Mit anderen Worten, wir können die Variable so deklarieren, als wäre sie von einem abstrakteren Typ, wie **List**, **Map** oder **Set**, anstatt eines konkreteren, wie **ArrayList**, **LinkedList**, **HashMap**, **TreeSet** usw. Zum Beispiel:

```
List<Track> tracks = new LinkedList<>();
```

oder

```
Map<String, String> antwortMap = new HashMap<>();
```

Hier ist die Variable als **List**-Typ deklariert, obwohl das erzeugte und gespeicherte Objekt vom Typ **LinkedList** ist (dasselbe gilt für **Map** und **HashMap**).

Variablen wie **tracks** und **antwortMap** in diesen Beispielen werden *polymorphe Variablen* genannt, weil sie in der Lage sind, auf Objekte mit unterschiedlichen, aber verwandten Typen zu verweisen. Die Variable **tracks** könnte entweder auf eine **ArrayList** oder auf eine **LinkedList** verweisen, während **antwortMap** auf ein **HashMap** oder ein **HashMap** verweisen könnte. Die grundlegende Idee, Variablen auf diese Weise zu deklarieren, besteht darin, dass die Art, wie die Variablen im restlichen Code verwendet werden, unabhängig vom exakten, konkreten Objekttyp ist, auf den sie verweisen. Ein Aufruf an **add**, **clear**, **get**, **remove** oder **size** auf **tracks** hätte überall dieselbe Syntax, und alle haben dieselbe Auswirkung auf die verknüpfte Sammlung. Der exakte Typ spielt selten jemals eine Rolle. Ein Vorteil der Verwendung polymorpher Variablen ist: Falls wir jemals von einem konkreten

Typ zu einem anderen wechseln wollen, muss der Code nur an einer einzigen Stelle geändert werden, nämlich dort, wo das Objekt erzeugt wird. Ein weiterer Vorteil ist, dass unser Code nicht den konkreten Typ kennen muss, der möglicherweise im Code, der von anderen Programmierern stammt, benutzt wird – wir müssen nur den abstrakten Typ kennen: **List**, **Map**, **Set** usw. Beide Vorteile *reduzieren die Kopplung* zwischen unterschiedlichen Teilen eines Programms, was eine gute Sache ist – wir werden dies in Kapitel 8 vertiefen.

Wir werden die Mechanismen hinter polymorphen Variablen ausführlich in den Kapiteln 10 und 11 untersuchen.

6.17.2 Die Methode `collect` aus Streams (fortgeschritten)

In diesem Abschnitt führen wir die Diskussion zu Streams und Lambda-Ausdrücken von Java 8 weiter, die wir in Kapitel 5 begonnen hatten. Dort hatten wir erklärt, dass es wichtig zu wissen ist, dass jede Stream-Operation den Eingabe-Stream unverändert lässt. Jede Operation erzeugt einen neuen Stream, basierend auf der Eingabe und der Operation, der an die nächste Operation in der Folge weitergegeben wird. Wir werden jedoch häufig Stream-Operationen einsetzen wollen, die eine neue Version der ursprünglichen Sammlung erzeugen, also eine veränderte Kopie – in der Regel durch Filtern der Originalinhalte, indem entweder nur die gewünschten Objekte ausgewählt oder diejenigen Elemente aussortiert werden, die nicht benötigt werden. Dazu müssen wir die Methode `collect` auf den Stream anwenden. Dies ist eine *terminale Operation* und wird daher als letzte Operation in einer Pipeline erscheinen. Sie kann eingesetzt werden, um die Elemente, die aus einer Stream-Operation resultieren, wieder einer neuen Operation zuzuführen.

Es gibt sogar zwei Versionen der `collect`-Methode, aber wir werden uns nur auf die einfachere konzentrieren. Tatsächlich werden wir unser Augenmerk nur darauf konzentrieren, wie wir ein neues Sammlungsobjekt als letzte Operation einer Pipeline erzeugen können, und überlassen weitere Untersuchungen dem interessierten Leser. Selbst die elementarste Nutzung bezieht mehrere Konzepte ein, die für dieses relativ frühe Stadium des Buchs schon fortgeschritten sind: Streams, statische Methoden und Polymorphismen.

Die `collect`-Methode hat als Parameter einen **Collector**. Dies ist im Wesentlichen etwas, das schrittweise die Elemente seines Eingabe-Streams irgendwie *akkumulieren* kann. Dies ähnelt der `reduce`-Operation auf Streams, die wir im vorigen Kapitel behandelt haben. Dort haben wir gesehen, dass ein Stream aus Zahlen auf eine einzige Zahl reduziert werden konnte. Hier werden wir einen Stream von Objekten auf ein einzelnes Objekt reduzieren, das eine weitere Sammlung ist. Da es uns häufig nicht besonders interessiert, ob eine Sammlung eine **ArrayList**, eine **LinkedList** oder irgendeine andere Form von Listtyp ist, die wir noch nicht kennengelernt haben, wir können den **Collector** anweisen, die Sammlung in der Form des polymorphen Typs **List** zurückzugeben.

Der einfachste Weg, einen **Collector** zu erhalten, ist eine der **static**-Methoden zu verwenden, die in der **Collectors**-Klasse im Paket **java.util.stream** definiert sind: `toList`, `toMap` und `toSet`.

Listing 6.9 zeigt eine Methode aus dem *Nationalpark*-Projekt, das wir uns in Kapitel 5 angesehen haben. Diese Methode filtert die vollständige List von **Sichtung**-Objekten, um eine neue **List** nur aus Sichtungen für ein bestimmtes Tier zu erzeugen. In unserer Version im *Tiermonitoring-v1*-Projekt wurde diese Methode im iterativen Stil mit einer **for-each**-Schleife geschrieben. Hier ist sie im funktionalen Stil unter Verwendung der **collect**-Methode formuliert.

Listing 6.9

Eine funktionale Version der Methode **gibSichtungUeber**.

```
/**
 * Gib eine Liste aller Sichtungen des angegebenen Tiers aus.
 * @param tier die Tierart
 * @return eine Liste aller Sichtungen des angegebenen Tiers
 */
public List<Sichtung> gibSichtungUeber(String tier)
{
    return sichtungen.stream()
        .filter(datensatz -> tier.equals(datensatz.gibTier()))
        .collect(Collectors.toList());
}
```

Ein heikler Punkt, den es zu beachten gilt, ist, dass die Methode **toList** von **Collectors** gibt kein echtes **List**-Objekt zur Weitergabe an die **collect**-Methode zurückgibt. Stattdessen liefert sie einen **Collector** zurück, der schließlich zur Erzeugung eines passenden konkreten **List**-Objekts führt.

Übung 6.85 Implementieren Sie diese Version der Methode **gibSichtungUeber** in Ihrem eigenen *Tiermonitoring*-Projekt.

Übung 6.86 Schreiben Sie die Methode **gibSichtungenInGebiet** aus dem ursprünglichen *Tiermonitoring*-Projekt im funktionalen Stil neu.

6.17.3 Methodenreferenzen (fortgeschritten)

Angenommen, wir wollten die Objekte aus einem gefilterten Stream in einem bestimmten Sammlungstyp wie einer **ArrayList** zusammenfassen, anstatt einen polymorphen Typ zu verwenden. Dazu muss der Ansatz, den wir im vorigen Abschnitt genutzt haben, abgewandelt werden. Es ist immer noch notwendig, der **collect**-Operation einen **Collector** zu übergeben, doch diesen müssen wir auf eine andere Weise erlangen. Listing 6.10 zeigt, wie dies gemacht werden könnte.

Listing 6.10

Eine weitere Version der Methode **gibSichtungUeber**.

```
/**
 * Gib eine Liste aller Sichtungen des angegebenen Tiers aus.
 * @param tier die Tierart
 * @return eine Liste aller Sichtungen des angegebenen Tiers
 */
public List<Sichtung> gibSichtungUeber(String tier)
{
    return sichtungen.stream()
        .filter(datensatz -> tier.equals(datensatz.gibTier()))
        .collect(Collectors.toCollection(ArrayList::new));
}
```

Dieses Mal haben wir die statische Methode **toCollection** von **Collectors** aufgerufen, die einen Parameter vom Typ **Supplier** besitzt. Der Parameter, den wir benutzt haben, führt eine Notation ein, die *Methodenreferenz* genannt wird, zu erkennen an den beiden aufeinanderfolgenden Doppelpunkten.

Methodenreferenzen sind eine bequeme Kurzschrift für einige Lambda-Ausdrücke. Sie werden häufig verwendet, wenn der Compiler auf den vollständigen Aufruf einer Methode einfach schließen kann, ohne dass diese im Detail explizit spezifiziert werden müssen. Es gibt vier Fälle von Methodenreferenzen:

- a** Eine Referenz auf einen Konstruktor, so wie wir es hier verwendet haben:

```
ArrayList::new
```

(Beachten Sie, dass streng genommen ein Konstruktor nicht wirklich eine Methode ist.) Dies ist eine Abkürzung für den Lambda-Ausdruck

```
() -> new ArrayList<>()
```

- b** Eine Referenz auf einen Methodenaufruf auf ein bestimmtes Objekt:

```
System.out::println
```

Dies wäre eine Abkürzung für den Lambda-Ausdruck:

```
str -> System.out.println(str)
```

Beachten Sie, das **System.out** eine **public static** Variable der Klasse **System** ist und der Parameter **str** würde durch den Kontext geliefert, in dem die Methodenreferenz verwendet wird, zum Beispiel als terminale Operation einer Stream-Pipeline:

```
forEach(System.out::println)
```

- c** Eine Referenz auf eine statische Methode:

```
Math::abs
```

Dies wäre eine Abkürzung für

```
x -> Math.abs(x)
```

Der Parameter **x** würde vom Kontext kommen, in dem die Methodenreferenz verwendet wird, zum Beispiel als Teil der Map-Operation eines Streams:

```
map(Math::abs)
```

- d** Eine Referenz auf eine Instanzmethode eines bestimmten Typs:

```
String::length
```

Dies wäre eine Abkürzung für:

```
str -> str.length()
```

Auch hier wird **str** wieder dem Kontext entnommen. Es ist wichtig, zwischen dieser Nutzungsart, bei der es um eine Instanzmethode geht, und Nutzung b), wo eine spezifische Instanz gemeint ist, sowie Nutzung c), die eine statische Methode umfasst, zu unterscheiden.

Wir haben hier nur die einfachsten Beispiele für den Einsatz von Methodenreferenzen aufgeführt. Komplexeren Fällen, die mehrere Parameter umfassen, wird man auch in den allgemeineren Beispielen begegnen.



Zusammenfassung

Der Umgang mit Bibliotheksklassen und Klassenschnittstellen ist essenziell für einen kompetenten Programmierer. Dies hat zwei Aspekte: Zum einen muss er die Beschreibung von Bibliotheksklassen lesen können (insbesondere ihre Schnittstellen), zum anderen muss er sie auch erstellen können.

Es ist wichtig, einige zentrale Klassen der Java-Standardklassenbibliothek zu kennen und zu wissen, wie man über weitere bei Bedarf etwas herausfinden kann. In diesem Kapitel haben wir einige der wichtigsten Klassen vorgestellt und gezeigt, wie man die Dokumentation der Bibliothek benutzt.

Es ist außerdem wichtig, in der Lage zu sein, jede selbst geschriebene Klasse im Stil der Bibliotheksklassen zu dokumentieren, damit andere Programmierer diese Klasse leicht benutzen können, ohne ihre Implementierung kennen zu müssen. Diese Dokumentation sollte gute Kommentare für jedes Projekt, jede Klasse und jede Methode enthalten. Mithilfe von **javadoc** können Sie solche Dokumentationen erzeugen.

NEUE BEGRIFFE IN DIESEM KAPITEL

Schnittstelle, Implementierung, Abbildung (Map), Menge (Set), Autoboxing, Wrapper-Klassen, **javadoc**, Zugriffsmodifikator, Geheimnisprinzip, Klassenvariable, Klassenmethode, **main**-Methode, **static**, Konstante, **final**, polymorphe Variable, Methodenreferenz

Zusammenfassung der Konzepte



- **Java-Bibliothek** Die Standardklassenbibliothek von Java enthält viele Klassen, die sehr nützlich sind. Es ist wichtig zu wissen, wie man die Bibliothek benutzen kann.
- **Dokumentation einer Bibliothek** Die Dokumentation der Standardbibliothek von Java zeigt Informationen über alle Klassen in der Bibliothek. Die Benutzung dieser Dokumentation ist unerlässlich für einen vernünftigen Gebrauch der Bibliotheksklassen.
- **Schnittstelle** Die Schnittstelle einer Klasse beschreibt, was eine Klasse leistet und wie sie benutzt werden kann, ohne dass ihre Implementierung sichtbar wird.
- **Implementierung** Der komplette Quelltext, der eine Klasse definiert, wird als die Implementierung dieser Klasse bezeichnet.
- **Unveränderlich** Ein Objekt wird als unveränderlich bezeichnet, wenn sein Inhalt oder Zustand nicht mehr geändert werden kann, nachdem es erzeugt wurde. **String** ist ein Beispiel für eine Klasse, die unveränderliche Objekte definiert.
- **Map** Eine Map ist eine Sammlung, die Schlüssel-Wert-Paare als Einträge enthält. Ein Wert kann ausgelesen werden, indem ein Schlüssel angegeben wird.
- **Set** Eine Menge (Set) ist eine Sammlung, in der jedes Element nur maximal einmal enthalten ist. Die Elemente einer Menge haben keine spezifische Ordnung.
- **Autoboxing** Autoboxing wird automatisch durchgeführt, wenn der Wert eines primitiven Typs in einem Kontext verwendet wird, in dem ein Wrapper-Typ erwartet wird.
- **Dokumentation** Die Dokumentation einer Klasse sollte genau die Informationen bieten, die andere Programmierer benötigen, um die Klasse ohne einen Blick auf ihre Implementierung benutzen zu können.
- **Zugriffsmodifikatoren** Zugriffsmodifikatoren definieren die Sichtbarkeit eines Datenfelds, eines Konstruktors oder einer Methode. Öffentliche (**public**) Elemente sind sowohl innerhalb der definierenden Klasse zugreifbar als auch von anderen Klassen aus; private (**private**) Elemente sind ausschließlich innerhalb der definierenden Klasse zugreifbar.
- **Geheimnisprinzip** Das Geheimnisprinzip besagt, dass die internen Details der Implementierung einer Klasse vor anderen Klassen verborgen sein sollten. Dies unterstützt eine bessere Modularisierung von Anwendungen.
- **Klassenvariablen, statische Variablen** Auch Klassen können Datenfelder haben. Diese werden Klassenvariablen oder auch statische Variablen genannt. Von einer Klassenvariablen existiert immer nur genau eine Kopie, unabhängig von der Anzahl der erzeugten Instanzen.

Übung 6.87 Im Internet geht das Gerücht um, dass George Lucas (der Schöpfer der Star-Wars-Filme) eine Formel benutzt, um Namen für die Charaktere in seinen Geschichten zu erzeugen (Jar Jar Binks, Obiwan Kenobi etc.). Diese Formel lautet – angeblich:

Ihr Star-Wars-Vorname:

- 1** Nehmen Sie die ersten drei Buchstaben Ihres Nachnamens.
- 2** Fügen Sie diesen die ersten beiden Buchstaben Ihres Vornamens hinzu.

Ihr Star-Wars-Nachname:

- 1** Nehmen Sie die ersten beiden Buchstaben vom Mädchennamen Ihrer Mutter.
- 2** Fügen Sie diesen die ersten drei Buchstaben Ihrer Geburtsstadt hinzu.

Ihre Aufgabe lautet nun: Erzeugen Sie ein BlueJ-Projekt namens *Star-Wars*. Legen Sie darin eine Klasse **Namensgenerator** an. Diese Klasse soll eine Methode **generiereStarWarsNamen** haben, die einen Namen nach dem oben angegebenen Schema produziert. Sie werden dazu die Methode **substring** der Klasse **String** benötigen, mit der Teile aus einer Zeichenkette herausgeschnitten werden können.

Übung 6.88 Die folgende Methode versucht, einen String in Großbuchstaben auszugeben:

```
public void inGrossbuchstabenAusgeben(String s)
{
    s.toUpperCase(); System.out.println(s);
}
```

Diese Methode funktioniert jedoch nicht. Finden Sie heraus, warum nicht, und erklären Sie es. Wie sollte es richtig aussehen?

Übung 6.89 Nehmen Sie an, wir wollen die Werte zweier **int**-Variablen **a** und **b** vertauschen. Dazu schreiben wir die Methode

```
public void tausche(int i1, int i2)
{
    int tmp = i1;
    i1 = i2;
    i2 = tmp;
}
```

Anschließend rufen Sie diese Methode mit unseren Variablen **a** und **b** auf:

```
tausche(a,b);
```

Sind **a** und **b** nach diesem Aufruf vertauscht? Wenn Sie es testen, werden Sie sehen, dass sie es nicht sind! Warum funktioniert dies nicht? Erklären Sie es detailliert.

KAPITEL

7

Sammlungen mit fester Größe – Arrays



Lernziele

Zentrale Konzepte in diesem Kapitel: Arrays, `for`-Schleife

Java-Konstrukte in diesem Kapitel: Array, `for`-Schleife, `++`, Konditionaloperator, `?:`

Der Hauptfokus dieses Kapitels ist die Verwendung von Array-Objekten für Sammlungen fester Größe und die `for`-Schleife.

7.1 Sammlungen fester Größe

In Kapitel 4 haben wir uns angesehen, in welchen Bibliotheksklassen, zum Beispiel **ArrayList**, wir Sammlungen von Objekten verwalten können. Wir sind in Kapitel 6 auch weiteren Sammlungstypen wie **HashSet** und **HashMap** begegnet. Ein wesentliches Merkmal all dieser Sammlungstypen ist ihre flexible Kapazität – wir müssen nie festlegen, wie viele Elemente eine konkrete Sammlung aufnehmen wird, und die Anzahl der gespeicherten Elemente kann während der Lebensdauer der Sammlung beliebig variieren.

In diesem Kapitel geht es nun um Sammlungen, die keine flexible, sondern eine feste Kapazität besitzen – zu dem Zeitpunkt, an dem das Sammlungsobjekt erzeugt wird, müssen wir die maximale Anzahl der Elemente angeben, die gespeichert werden können, und diese Anzahl kann auch nicht mehr geändert werden. Das mag zunächst wie eine unnötige Einschränkung erscheinen. Es gibt allerdings Situationen, in denen wir im Voraus wissen, wie viele Elemente wir in einer Sammlung ablegen wollen, und diese Anzahl bleibt dann über die Lebenszeit der Sammlung konstant. Für solche Situationen haben wir die Möglichkeit, eine Sammlung mit einer festen Größe zum Speichern der Objekte zu wählen.

7.2 Arrays

Eine Sammlung mit einer festen Größe wird *Array* genannt. Obwohl die feste Größe bei Arrays in vielen Situationen ein deutlicher Nachteil sein kann, haben sie gegenüber Sammlungen mit flexibler Größe mindestens zwei Vorteile:

- Der Zugriff auf die Elemente in einem Array ist häufig sehr viel effizienter als auf die Elemente einer vergleichbaren Sammlung mit flexibler Größe.

Konzept

Ein **Array** ist eine besondere Art von Sammlung, die eine festgelegte Anzahl von Elementen halten kann.